School of Computing

Department of Computer Science and Engineering

ACADEMIC YEAR: 2025-26

(WINTER SEMESTER)

LAB RECORD

10211CS224 - Full Stack Application Development

NAME :K.Pallavi

VTU.NO :26490

REG.NO :23UECS1103

BRANCH :CSE(AI&DS)

YEAR/SEM :3rd /6th sem

SLOT :E1

10211CS224 - Full Stack Application Development

TABLE OF CONTENTS

Task-1: Student Registration & Data Storage System

AIM: To develop a Student Registration System that allows users to enter, store, and manage student details efficiently using a web interface and a MySQL database.

ALGORITHM:

Step 1: Start

Step 2: Open the web application (index page)

Step 3: Display home page with option to go to registration page

Step 4: User navigates to the registration form

Step 5: User enters student details (like name, ID, course, etc.)

Step 6: Validate input fields

- Check if fields are empty
- Ensure correct data format

Step 7: Submit the form data to the server

Step 8: Server receives the data using Node.js

Step 9: Establish connection with MySQL database

- If connection fails → display error
- Else → proceed

Step 10: Insert student data into database table

Step 11: Confirm successful storage of data

Step 12: Display success message to user

Step 13: Allow user to view/manage stored data (if implemented)

Step 14: Stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
```

```
<html>

<head>

<title>Student Portal</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="home-container">

<h1>Welcome to Student Portal</h1>

<p>Register and manage student information easily.</p>

<a href="registration.html" class="start-btn">Go to Registration</a>

</div>

</body>

</html>
```

REGISTRATION.HTML:

```
<!DOCTYPE html>

<html>

<head>

<title>Student Registration</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<h1>Student Registration</h1>

<button onclick="goHome()" class="home-btn">Home</button>

<form id="studentForm">

<label>Name</label>
```

```
<input type="text" id="name" placeholder="Enter your name" required>
<label>Email</label>
<input type="email" id="email" placeholder="Enter your email" required>
<label>Date of Birth</label>
<input type="date" id="dob" required>
<label>Department</label>
<select id="department" required>
<option value="">Select Department</option>
<option>CSE</option>
<option>IT</option>
<option>ECE</option>
<option>MECH</option>
</select>
<label>Phone</label>
<input type="text" id="phone" placeholder="Enter phone number" required>
<button type="submit">Register</button>
</form>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{ font-family: 'Segoe UI', sans-serif;
background: linear-
gradient(135deg,#0f2027,#203a43,#2c5364); height:100vh;
display:flex; justify-
content:center; align-
```

```
items:center;
margin:0;
}
.container{ width:420px; padding:35px;
background:rgba(255,255,255,0.08);
border-radius:18px; backdrop-filter:
blur(10px); box-shadow:0 10px 35px
rgba(0,0,0,0.6);
}
h1{ text-align:center;
color:white; margin-
bottom:25px;
}
h2{ text-
align:center;
color:white; margin-
top:30px;
}
form{ display:flex;
flex-direction:column;
}
label{ color:#ddd;
font-size:14px;
margin-
```

```
bottom:5px;
margin-top:10px;
}
input,select{
width:100%;
padding:12px;
border:none; border-
radius:8px;
background:#27496d;
color:white; font-
size:14px;
outline:none; margin-
bottom:5px;
}
input:focus,select:focus{ box-
shadow:0 0 8px #00eaff;
background:#2c5d8a;
}
button{ margin-
top:15px;
padding:12px;
border:none; border-
radius:10px;
background:linear-
gradient(45deg,#ff4b
2b,#ff416c);
```

```
color:white; font-
size:16px; font-
weight:bold;
cursor:pointer;
transition:0.3s;
}
button:hover{
transform:scale(1.05); box-
shadow:0 0 15px #ff416c;
}
#studentList{ margin-
top:15px; text-
align:center;
color:white; font-
size:15px;
}
.home-container{ text-align:center;
color:white;
background:rgba(255,255,255,0.08);
padding:50px; border-radius:20px;
backdrop-filter: blur(10px); box-
shadow:0 10px 35px rgba(0,0,0,0.6);
width:420px;
}
```

```
.home-container h1{
font-size:32px;
margin-bottom:15px;
}
.home-container p{
margin-bottom:30px;
color:#ddd;
}
.start-btn{ text-decoration:none; padding:12px
25px; border-radius:10px; background:linear-
gradient(45deg,#ff4b2b,#ff416c); color:white; font-
weight:bold; transition:0.3s;
}
.start-btn:hover{
transform:scale(1.05); box-
shadow:0 0 15px #ff416c;
}
```

SCRIPT.JS:

```
const form = document.getElementById("studentForm") form.addEventListener("submit", async (e) => {
e.preventDefault() const student = { name:
document.getElementById("name").value, email:
document.getElementById("email").value, dob:
document.getElementById("dob").value, department:
```

```
document.getElementById("department").value,        phone:
document.getElementById("phone").value
}
try{ const res = await
fetch("http://localhost:3000/addStudent",{
method:"POST", headers:{
"Content-Type":"application/json"
},
body:JSON.stringify(student)
})
const data = await
res.text() alert(data) //
clear form form.reset()
}catch(error){
console.log("Error:",error)
}
})
function goHome(){
window.location.href="index.html"
}
```

SERVER.JS: const express =

```
require("express") const cors =
require("cors") const path =
require("path") const db =
require("./db") const app =
```

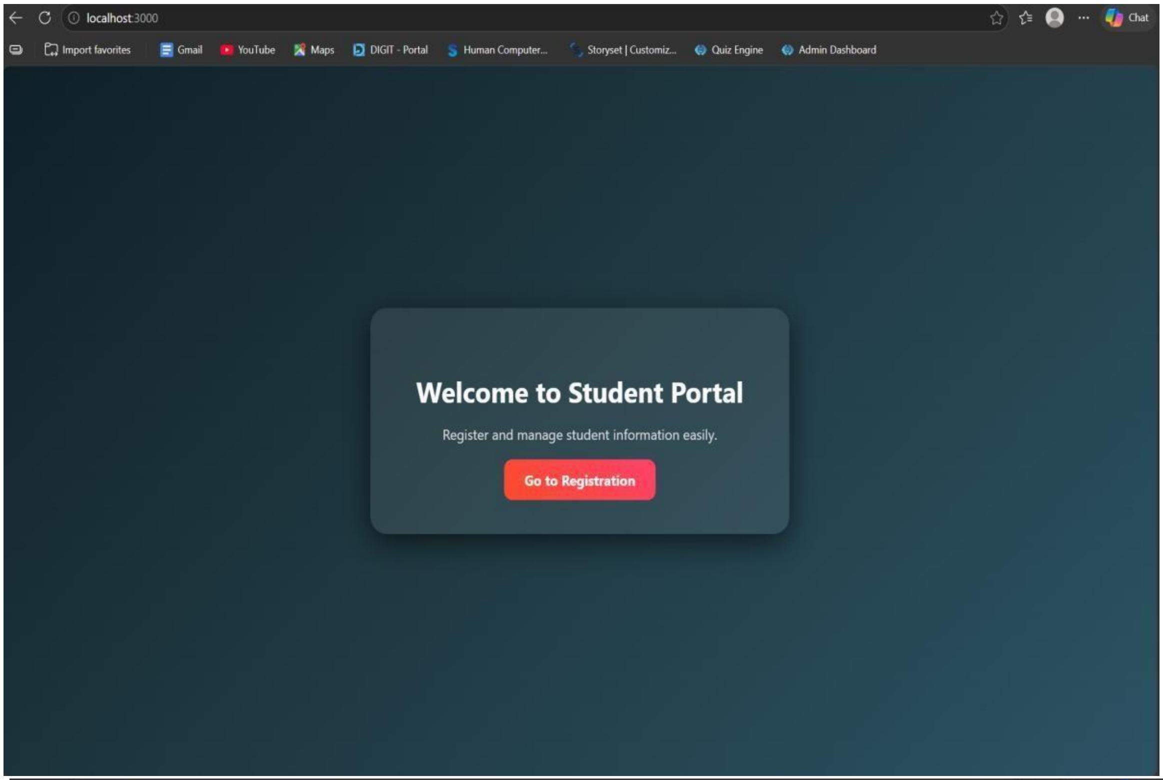
```
express()                app.use(cors())
app.use(express.json())
// Serve static files (HTML, CSS, JS)
app.use(express.static(_dirname)) // Redirect root
link to index.html app.get("/", (req, res) => {
res.sendFile(path.join(_dirname, "index.html"))
})
// Insert student data app.post("/addStudent", (req, res) => { const
{name,email,dob,department,phone} = req.body const sql = "INSERT INTO
students(name,email,dob,department,phone) VALUES (?,?,?,?,?)"
db.query(sql,[name,email,dob,department,phone],(err,result)=>{
if(err){ res.send(err) }else{
res.send("Student Added
Successfully") }

})


})
// Get student data
app.get("/students",(req,res)=>{
db.query("SELECT * FROM
students",(err,result)=>{
if(err){
res.send(err)
}else{
res.json(result)
```

```
}
})


})
//          Start          server          app.listen(3000,()=>{
console.log("Server                running                at:
http://localhost:3000")
})
```

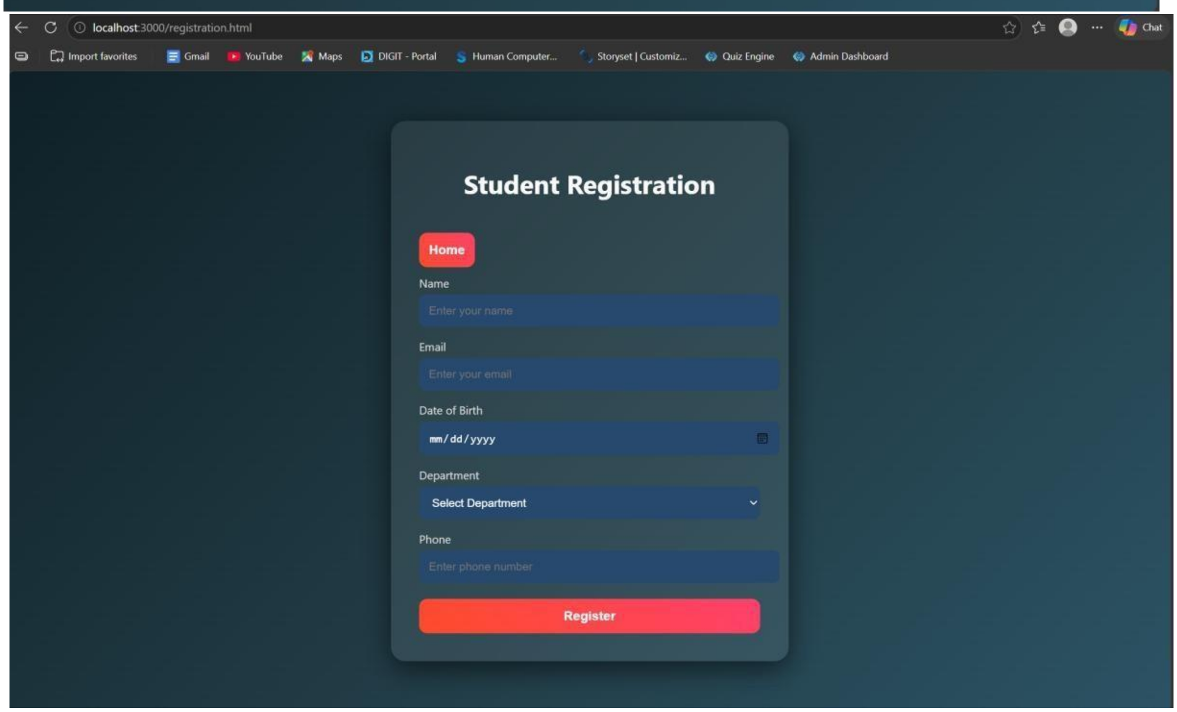
DB.JS:

```
const mysql = require("mysql") const
db = mysql.createConnection({ host:
"localhost", user: "root", password:
"fahi",

database: "studentdb"
})
db.connect((err)=>{
if(err){        console.log("Database
Error:", err)
}else{       console.log("MySQL       Connected
Successfully")
}
})
module.exports = db
```

OUTPUTS:



localhost:3000
Import favorites
Gmail
YouTube
Maps
DIGIT - Portal
Human Computer...
Storyset | Customiz...
Quiz Engine
Admin Dashboard
Chat
Welcome to Student Portal
Register and manage student information easily.
Go to Registration



localhost:3000/registration.html
Import favorites
Gmail
YouTube
Maps
DIGIT - Portal
Human Computer...
Storyset | Customiz...
Quiz Engine
Admin Dashboard
Chat
Student Registration
Home
Name
Enter your name
Email
Enter your email
Date of Birth
mm/dd/yyyy
Department
Select Department
Phone
Enter phone number
Register

RESULT: The student registration system was successfully developed to store and manage student data using a web interface and MySQL database**.**

TASK2: **Data Retrieval & Sorting Dashboard**

AIM: To develop a web-based dashboard that allows users to store, retrieve, and display student data efficiently from a database, and present it in a structured format for easy access and management.

ALGORITHM:

1. Start the application
2. Initialize the server using Express framework
3. Establish database connection using MySQL 4. Load frontend pages (HTML, CSS, JS) in browser

➤ Data Insertion Process:

5. User enters student details (Name, Email, DOB, Department, Phone)
6. Send data to backend using POST request (/addStudent)
7. Execute SQL query to insert data into database
8. Display success message after insertion

➤ Data Retrieval Process:

9. Send GET request to server (/students)
10. Fetch all student records from database
11. Convert data into JSON format
12. Display data dynamically in dashboard table.
13. Arrange data in structured format (table/grid)
14. Allow better readability and management
15. Stop.

PROGRAM:

INDEX.HTML:

```html
<!DOCTYPE html>

<html>

<head>

<title>Student Portal</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="home-container">

<h1>Welcome to Student Portal</h1>

<p>Register and manage student information easily.</p>

<a href="registration.html" class="start-btn">Go to Registration</a>

<a href="dashboard.html" class="start-btn">View Dashboard</a>

</div>

</body>

</html>
```

REGISTRATION.HTML:

```html
<!DOCTYPE html>

<html>

<head>

<title>Student Registration</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<h1>Student Registration</h1>
```

```
<button onclick="goHome()" class="home-btn">Home</button>

<form id="studentForm">

<label>Name</label>

<input type="text" id="name" placeholder="Enter your name" required>

<label>Email</label>

<input type="email" id="email" placeholder="Enter your email" required>

<label>Date of Birth</label>

<input type="date" id="dob" required>

<label>Department</label>

<select id="department" required>

<option value="">Select Department</option>

<option>CSE</option>

<option>IT</option>

<option>ECE</option>

<option>MECH</option>

</select>

<label>Phone</label>

<input type="text" id="phone" placeholder="Enter phone number" required>

<button type="submit">Register</button>

</form>

</div>

<script src="script.js"></script>

</body>

</html>
```

DASHBOARD.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Student Dashboard</title>
<link rel="stylesheet" href="dashboard.css">
</head>
<body>
<div class="container">
<button onclick="goHome()" class="home-btn">Home</button>
<h1>Student Dashboard</h1>
<div class="controls">
<select id="sortSelect">
<option value="">Sort By</option>
<option value="name">Name</option>
<option value="dob">Date of Birth</option>
</select>
<select id="departmentFilter">
<option value="">Filter by Department</option>
<option value="CSE">CSE</option>
<option value="IT">IT</option>
<option value="ECE">ECE</option>
<option value="MECH">MECH</option>
</select>
<button onclick="loadStudents()">Apply</button>
</div>
<table>
```

```
<thead>
<tr>
<th>ID</th>
<th>Name</th>
<th>Email</th>
<th>DOB</th>
<th>Department</th>
<th>Phone</th>
</tr>
</thead>
<tbody id="studentTable"></tbody>
</table>
<h2>Department Count</h2>
<div id="deptCount"></div>
</div>
<script src="dashboard.js"></script>
<script> function goHome(){
window.location.href =
"index.html"
}
</script>
</body>
</html>
```

STYLE.CSS: body{ font-family: 'Segoe UI', sans-serif; background:

linear-gradient(135deg,#0f2027,#203a43,#2c5364);

```
height:100vh;    display:flex;    justify-content:center;    align-
items:center; margin:0;
}

.container{  width:420px;  padding:35px;
background:rgba(255,255,255,0.08);
border-radius:18px;       backdrop-filter:
blur(10px);   box-shadow:0   10px   35px
rgba(0,0,0,0.6);
}

h1{ text-align:center;
color:white; margin-
bottom:25px;
}

h2{ text-
align:center;
color:white; margin-
top:30px; }

form{ display:flex;
flex-direction:column;
}
```

```
label{ color:#ddd;

font-size:14px;

margin-

bottom:5px;

margin-top:10px;

}


input,select{

width:100%;

padding:12px;

border:none; border-

radius:8px;

background:#27496d;

color:white; font-

size:14px;

outline:none; margin-

bottom:5px;

}

input:focus,select:foc

us{ box-shadow:0 0

8px #00eaff;

background:#2c5d8a;

}
```

```
button{ margin-top:15px; padding:12px;
border:none; border-radius:10px;
background:linear-
gradient(45deg,#ff4b2b,#ff416c); color:white; font-
size:16px; font-weight:bold; cursor:pointer;
transition:0.3s;
}

button:hover{
transform:scale(1.05); box-
shadow:0 0 15px #ff416c;
}

#studentList{ margin-
top:15px; text-
align:center;
color:white; font-
size:15px;
}
.home-container{         text-align:center;
color:white;
background:rgba(255,255,255,0.08);
padding:50px;           border-radius:20px;
backdrop-filter:        blur(10px);        box-
```

```
shadow:0   10px   35px   rgba(0,0,0,0.6);
width:420px;
}

.home-container h1{
font-size:32px;
margin-bottom:15px;
}

.home-container p{
margin-bottom:30px;
color:#ddd;
}

.start-btn{ text-
decoration:none;
padding:12px 25px;
border-radius:10px;
background:linear-
gradient(45deg,#ff4b
2b,#ff416c);
color:white; font-
weight:bold;
transition:0.3s;
}
```

.start-btn:hover{

transform:scale(1.05); box-

shadow:0 0 15px #ff416c;

}

DASHBOARD.CSS: body{ font-family:Segoe UI;

background:linear-

gradient(135deg,#0f2027,#203a43,#2c5364); margin:0;

padding:0; display:flex; justify-content:center;

}

.container{

width:900px; margin-

top:40px;

background:rgba(255,

255,255,0.08);

padding:30px;

border-radius:15px;

backdrop-

filter:blur(10px);

color:white;

}

```
h1{ text-align:center;
margin-
bottom:20px;
}

.controls{ display:flex;
gap:10px; margin-
bottom:20px;
}

select,button{
padding:10px;
border:none; border-
radius:6px;
}

button{
background:#ff416c;
color:white;
cursor:pointer;
}

table{ width:100%; border-
collapse:collapse;
```

```
background:rgba(255,255,255,0.1)
;
}

th,td{
padding:10px; text-
align:center;
}

th{
background:#1f4068;
}

tr:nth-child(even){
background:rgba(255,255,255,0.05);
}

#deptCount{ margin-
top:15px; font-
size:16px;
}
.home-btn{
padding:8px 16px;
border:none; border-
radius:6px;
```

```
background:#00c6ff;

color:white;

cursor:pointer;

margin-bottom:10px;

}
```

SCRIPT.JS:

```
const form = document.getElementById("studentForm") form.addEventListener("submit",

async (e) => {

e.preventDefault()      const      student      =      {      name:

document.getElementById("name").value,                email:

document.getElementById("email").value,                   dob:

document.getElementById("dob").value,          department:

document.getElementById("department").value,       phone:

document.getElementById("phone").value

}

try{ const res = await

fetch("http://localhost:3000/addStudent",{ method:"POST",

headers:{

"Content-Type":"application/json"

},

body:JSON.stringify(student)

})

const data = await

res.text() alert(data) //

clear form form.reset()
```

```
}catch(error){
console.log("Error:",error)
}
})
function goHome(){
window.location.href="index.html"
}
```

SERVER.JS:

```
const express = require("express")
const cors = require("cors") const
path = require("path") const db =
require("./db") const app =
express() app.use(cors())
app.use(express.json()) // Serve
static files (HTML, CSS, JS)
app.use(express.static(__dirname))
// Redirect root link to index.html
app.get("/", (req, res) => {
res.sendFile(path.join(___dirname,
"index.html"))
})
// Insert student data app.post("/addStudent", (req, res) => { const
{name,email,dob,department,phone} = req.body const sql = "INSERT INTO
students(name,email,dob,department,phone) VALUES (?,?,?,?,?)"
db.query(sql,[name,email,dob,department,phone],(err,result)=>{
```

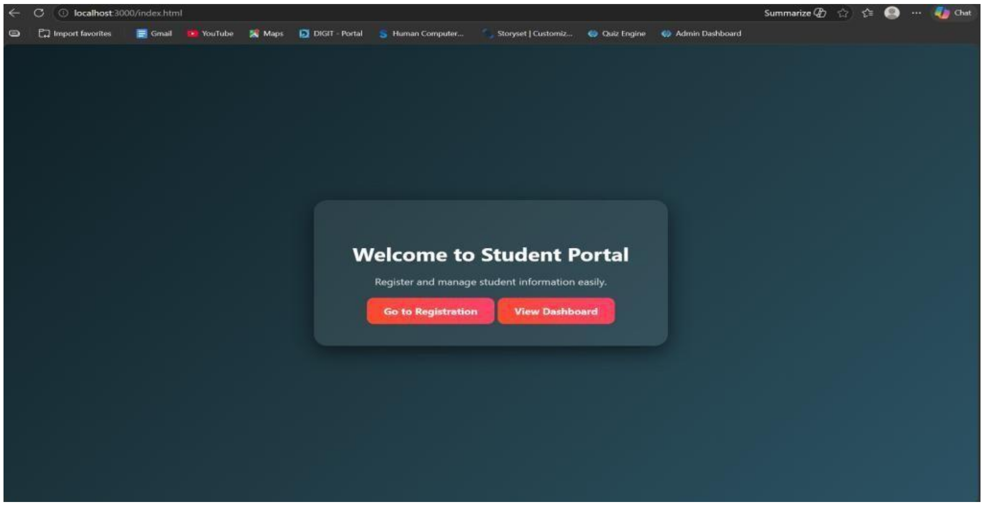
```
if(err){ res.send(err) }else{
res.send("Student Added
Successfully")
}
})


})
//		Get		student		data
app.get("/students",(req,res)=>{ db.query("SELECT
* FROM students",(err,result)=>{
if(err){
res.send(err)
}else{
res.json(result)
}
})
})
//	Start	server	app.listen(3000,()=>{
console.log("Server		running		at:
http://localhost:3000")
})
```

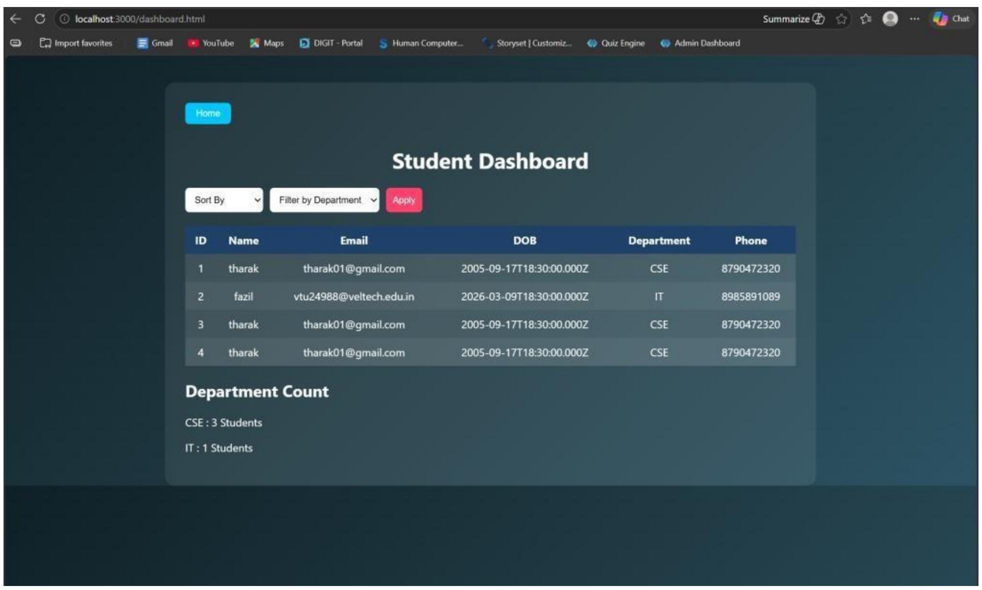
DB.JS:

```
const mysql = require("mysql")
const db = mysql.createConnection({
host: "localhost", user: "root",
```

```
password:       "fahi",       database:
"studentdb"
})
db.connect((err)=>{
if(err){       console.log("Database
Error:", err)
}else{      console.log("MySQL      Connected
Successfully")
}
})
module.exports = db
```

OUTPUTS:





RESULT: The system successfully stores, retrieves, and displays student data efficiently through a user-friendly dashboard.

TASK3: : Login System with Validation

AIM: To develop a secure login system that validates user credentials (username and password) and allows access only to authorized users.

ALGORITHM:

1. Start the application
2. Display login form (Username & Password fields)
3. User enters login credentials
4. Perform input validation:

    ○ Check if fields are empty ○ Validate email format (if applicable) ○ Ensure password meets required criteria

5. Send login data to backend server
6. Server receives request and processes input 7. Compare entered credentials with database records
8. If credentials match:

    ○ Grant access and redirect to dashboard

9. Else: ○ Display error message (“Invalid Username/Password”)
10. End process

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Student Portal</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="home-container">
```

```
<h1>Welcome to Student Portal</h1>
<p>Register and manage student information easily.</p>
<a href="registration.html" class="start-btn">Go to Registration</a>
<a href="dashboard.html" class="start-btn">View Dashboard</a>
</div>
</body>
</html>
```

DASHBOARD.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Student Dashboard</title>
<link rel="stylesheet" href="dashboard.css">
</head>
<body>
<div class="container">
<button onclick="goHome()" class="home-btn">Home</button>
<h1>Student Dashboard</h1>
<div class="controls">
<select id="sortSelect">
<option value="">Sort By</option>
<option value="name">Name</option>
<option value="dob">Date of Birth</option>
</select>
<select id="departmentFilter">
```

```
<option value="">Filter by Department</option>
<option             value="CSE">CSE</option>
<option value="IT">IT</option>
<option value="ECE">ECE</option>
<option value="MECH">MECH</option>
</select>
<button onclick="loadStudents()">Apply</button>
</div>
<table>
<thead>
<tr>
<th>ID</th>
<th>Name</th>
<th>Email</th>
<th>DOB</th>
<th>Department</th>
<th>Phone</th>
</tr>
</thead>
<tbody id="studentTable"></tbody>
</table>
<h2>Department Count</h2>
<div id="deptCount"></div>
</div>
<script src="dashboard.js"></script>
```

```
<script> function goHome(){
window.location.href =
"index.html"
}
</script>
</body>
</html>
```

LOGIN.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Login</title>
<link rel="stylesheet" href="login.css">
</head>
<body>
<div class="login-container">
<h1>Login</h1>
<form id="loginForm">
<label>Username</label>
<input type="text" id="username"> <label>Password</label>
<input type="password" id="password">
<p id="errorMsg"></p>
<button type="submit">Login</button>
</form>
</div>
```

```
<script src="login.js"></script>
</body>
</html>
```

REGISTRATION.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Student Registration</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h1>Student Registration</h1>
<button onclick="goHome()" class="home-btn">Home</button>
<form id="studentForm">
<label>Name</label>
<input type="text" id="name" placeholder="Enter your name" required>
<label>Email</label>
<input type="email" id="email" placeholder="Enter your email" required> <label>Date of Birth</label>
<input type="date" id="dob" required>
<label>Department</label>
<select id="department" required>
<option value="">Select Department</option>
```

```html
<option>CSE</option>
<option>IT</option>
<option>ECE</option>
<option>MECH</option>
</select>
<label>Phone</label>
<input type="text" id="phone" placeholder="Enter phone number" required>
<button type="submit">Register</button>
</form>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```css
body{ font-family: 'Segoe UI', sans-serif;
background: linear-
gradient(135deg,#0f2027,#203a43,#2c5364); height:100vh;
display:flex; justify-content:center;
align-items:center;
margin:0;
}

.container{  width:420px;  padding:35px;
background:rgba(255,255,255,0.08);
border-radius:18px;        backdrop-filter:
```

```
blur(10px);   box-shadow:0   10px   35px
rgba(0,0,0,0.6);
}

h1{ text-align:center;
color:white; margin-
bottom:25px;
}

h2{ text-
align:center;
color:white;
margin-top:30px;
}

form{
display:flex;                              flex-
direction:column;
}

label{ color:#ddd;
font-size:14px;
margin-
bottom:5px;
margin-top:10px;
}
```

```
input,select{
width:100%;
padding:12px;
border:none; border-
radius:8px;
background:#27496d;
color:white; font-
size:14px; outline:none;
margin-bottom:5px;
}

input:focus,select:focus{ box-
shadow:0 0 8px #00eaff;
background:#2c5d8a;
}

button{ margin-top:15px; padding:12px;
border:none; border-radius:10px;
background:linear-
gradient(45deg,#ff4b2b,#ff416c); color:white; font-
size:16px; font-weight:bold; cursor:pointer;
transition:0.3s;
}
```

```
button:hover{
transform:scale(1.05); box-
shadow:0 0 15px #ff416c;
}

#studentList{
margin-top:15px;
text-align:center;
color:white; font-
size:15px;
}
.home-container{         text-align:center;
color:white;
background:rgba(255,255,255,0.08);
padding:50px;          border-radius:20px;
backdrop-filter:       blur(10px);       box-
shadow:0    10px    35px    rgba(0,0,0,0.6);
width:420px;
}

.home-container h1{ font-
size:32px; margin-bottom:15px;
}
```

.home-container p{ margin-

bottom:30px; color:#ddd;

}

.start-btn{ text-decoration:none; padding:12px

25px; border-radius:10px; background:linear-

gradient(45deg,#ff4b2b,#ff416c); color:white; font-

weight:bold; transition:0.3s;

}

.start-btn:hover{

transform:scale(1.05); box-

shadow:0 0 15px #ff416c;

}

SERVER.JS:

const express = require("express") const cors = require("cors") const path = require("path") const db = require("./db") const app = express() app.use(cors()) app.use(express.json()) app.use(express.urlencoded({ extended: true })) // Disable default index.html loading app.use(express.static(_dirname, { index: false }))

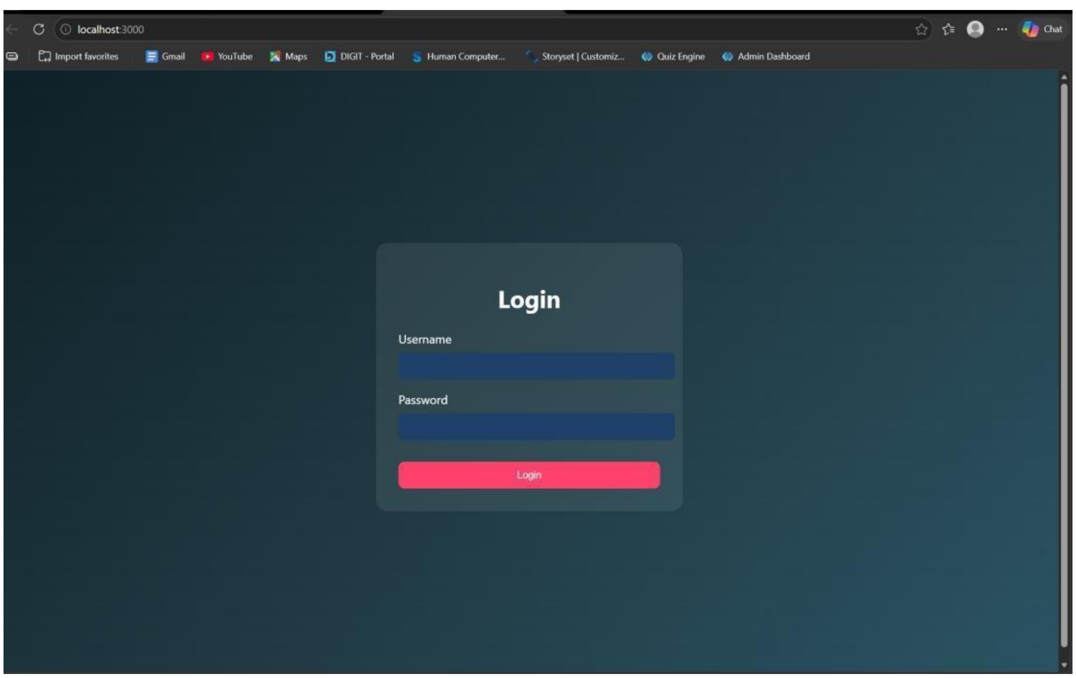
// =========================

```
// OPEN LOGIN PAGE FIRST //
=========================
app.get("/", (req, res) => {
res.sendFile(path.join(_dirname, "login.html"))
})

// =========================
// LOGIN SYSTEM
// ========================= app.post("/login",(req,res)=>{ const
{username,password} = req.body const sql = "SELECT * FROM users
WHERE username=? AND password=?"
db.query(sql,[username,password],(err,result)=>{
if(err){
res.send("error")
return
}
if(result.length > 0){
res.send("success")
}else{
res.send("fail")
}
})
})
// =========================
// STUDENT REGISTRATION
```

```
// ========================== app.post("/addStudent",(req,res)=>{ const {name,email,dob,department,phone} = req.body const sql = "INSERT INTO students(name,email,dob,department,phone) VALUES (?,?,?,?,?)" db.query(sql,[name,email,dob,department,phone],(err,result)=>{

if(err){ res.send(err) }else{
res.send("Student Added
Successfully")
}


})


})
// ==========================
// GET STUDENTS (Dashboard)
// ==========================

app.get("/students",(req,res)=>{

db.query("SELECT * FROM students",(err,result)=>{

if(err){
res.send(err)
}else{
res.json(result)
}
```

```
})


})
// ==========================
// START SERVER
//                              ==========================
app.listen(3000,()=>{  console.log("Server  running  at:
http://localhost:3000")
})
```

OUTPUTS:



RESULT: The system successfully validates user credentials and grants access only to authorized users while preventing invalid logins.

TASK4: Order Management using Joins

AIM: To design and implement an order management system using SQL joins and subqueries to efficiently retrieve and analyze customer, product, and order data.

ALGORITHM:

1. Start the process
2. Create database tables:

   ○ Customers (CustomerID, Name, etc.) ○ Products (ProductID, ProductName, Price) ○ Orders (OrderID, CustomerID, ProductID, Quantity, TotalAmount)

3. Insert sample data into all tables

➤ Data Retrieval using Joins:

4. Use SQL JOIN queries to combine data from Customers, Orders, and Products 5. Display customer order history with details (Name, Product, Quantity, Total Amount)

➤ Subquery for Analysis:

6. Use subquery to find the highest value order
7. Use subquery with GROUP BY to find the most active customer (maximum number of orders)

➤ Sorting:

8. Apply ORDER BY clause to sort results (e.g., highest order value first)
9. Display results in structured format (table/UI with CSS if frontend used)
10. Stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Login</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="login">
<h2>Admin Login</h2>
<input type="text" id="username" placeholder="Username">
<input type="password" id="password" placeholder="Password">
<button onclick="login()">Login</button>
</div>
<script src="script.js"></script>
</body>
</html>
```

ADMINDASHBOARD.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Order Dashboard</title>
<link rel="stylesheet" href="style.css">
</head>
```

```html
<body>

<!-- NAVBAR -->

<div class="navbar">

<h2>Order Management</h2>

<a href="index.html">

<button class="logout-btn">Logout</button>

</a>

</div>

<h1>Order Management Dashboard</h1>

<!-- CARDS SECTION -->

<div class="grid">

<!-- Add Customer -->

<div class="card">

<h3>Add Customer</h3>

<input id="cname" placeholder="Customer Name">

<input id="cemail" placeholder="Email">

<button onclick="addCustomer()">Add Customer</button>

</div>

<!-- Add Product -->

<div class="card">

<h3>Add Product</h3>

<input id="pname" placeholder="Product Name">

<input id="pprice" placeholder="Price">

<button onclick="addProduct()">Add Product</button>

</div>

<!-- Place Order -->
```

```
<div class="card">
<h3>Place Order</h3>
<select id="customerSelect">
<option value="">Select Customer</option>
</select>
<select id="productSelect">
<option value="">Select Product</option>
</select>
<input id="qty" placeholder="Quantity">
<button onclick="placeOrder()">Place Order</button>
</div>
</div>
<!-- ACTION BUTTONS -->
<div class="actions">
<button onclick="loadOrders()">Order History</button>
<button onclick="highestOrder()">Highest Order</button>
<button onclick="activeCustomer()">Most Active Customer</button>
</div>
<!-- RESULT AREA -->
<div id="result"></div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
*{ margin:0; padding:0; box-sizing:border-box; font-family:

'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

}


body{ background: linear-

gradient(135deg,#5f6caf,#9a6bff); min-

height:100vh; display:flex; flex-direction:column;

align-items:center;

}


/* LOGIN PAGE */


.login{ width:350px; padding:40px;

margin-top:120px; background:white;

border-radius:15px; box-shadow:0 10px

25px rgba(0,0,0,0.2); text-align:center;

}
.login h2{ margin-

bottom:20px;

color:#333;

}


.login                input{

width:100%;

padding:12px;

margin:10px           0;
```

```
border:1px solid
#ccc; border-
radius:8px;
outline:none;
}

.login button{
width:100%;
padding:12px;
border:none;
background:#5f6caf;
color:white; font-
weight:bold; border-
radius:8px;
cursor:pointer;
transition:0.3s;
}
.login button:hover{ background:#7c4dff;
}

/* DASHBOARD TITLE */

h1{ margin-top:30px;
margin-
bottom:25px;
color:white;
```

```
}

/* CARD GRID */

.grid{ display:flex;
gap:25px; flex-
wrap:wrap; justify-
content:center;
margin-bottom:20px;
}

/* CARDS */

.card{
width:260px; padding:22px; border-
radius:15px; box-shadow:0 8px 20px
rgba(0,0,0,0.2); text-align:center;
transition:0.3s; color:white;
}

/* Different card colors */

.card:nth-child(1){                    background:linear-
gradient(135deg,#ff7a18,#ffb347);
}
```

```
}

.card:nth-child(2){                                   background:linear-
gradient(135deg,#00c6ff,#0072ff);
}

.card:nth-child(3){ background:linear-
gradient(135deg,#43e97b,#38f9d7); color:#222;
}

.card:hover{ transform:translateY(-
6px); }

.card      h3{      margin-
bottom:10px;
}

/* INPUTS */

.card input{
width:90%;
padding:10px;
margin:6px; border-
radius:6px;
border:none;
}
```

```
/* CARD BUTTON */

.card           button{
padding:10px
15px;
border:none;
background:white;
color:#333;
border-radius:6px;
cursor:pointer;
font-weight:bold;
transition:0.3s;
}

.card button:hover{ background:#f1f1f1;
}

/* ACTION BUTTONS */

button{
margin:10px;
padding:12px 18px;
border:none;
border-radius:6px;
```

```
background:#ffffff;
color:#333; font-
weight:bold;
cursor:pointer;
transition:0.3s;
}

button:hover{
background:#eaeaea;
}

/* TABLE */
table{ width:85%; margin:auto;
margin-top:30px; border-
collapse:collapse; background:white;
border-radius:12px; overflow:hidden;
box-shadow:0 5px 15px
rgba(0,0,0,0.2);
}

th{
background:#5f6caf;
color:white;
padding:14px; font-
size:18px;
```

```
}

td{ padding:12px; text-
align:center; border-
bottom:1px solid #ddd;
color:#222; font-weight:500;
}
tr:nth-child(even){
background:#f7f7f7;
}

tr:hover{
background:#eef1ff;
}

/* RESULT TEXT */

#result{ margin-
top:20px; color:white;
font-size:18px; text-
align:center;
}
/* NAVBAR */

.navbar{
```

```
width:100%;
display:flex; justify-
content:space-between; align-
items:center; padding:15px
40px; background:#2c2f5c;
color:white;
box-shadow:0 4px 10px rgba(0,0,0,0.2);

}

.navbar      h2{      font-
weight:600;
}

.logout-btn{

background:#ff5252;
color:white;
border:none;
padding:10px 16px;
border-radius:6px;
cursor:pointer;
font-weight:bold;
```

```
}

.logout-btn:hover{

background:#ff1744;
}
.actions{

display:flex; justify-
content:center;
gap:20px; margin-
top:30px; flex-
wrap:wrap;

}
.actions        button{
padding:12px  20px;
font-size:16px;
border:none;
border-radius:8px;
background:#5f6caf
;           color:white;
cursor:pointer;
transition:0.3s;
}
```

```
.actions
button:hover{
background:#7c4dff;
transform:scale(1.05);

}
```

SCRIPT.JS:

```
// LOGIN FUNCTION

function login(){

const      username=document.getElementById("username").value
const password=document.getElementById("password").value

fetch("/login",{

method:"POST",
headers:{"Content-Type":"application/json"},

body:JSON.stringify({username,password})

})

.then(res=>res.json())
```

```
.then(data=>{

if(data.success){

window.location="admindashboard.html"
}else{ alert("Invalid
login") }

})

}
// ADD CUSTOMER

function addCustomer(){

fetch("/add-customer",{

method:"POST",
headers:{"Content-Type":"application/json"},

body:JSON.stringify({

name:document.getElementById("cname").value,
email:document.getElementById("cemail").value

})
```

```
})

.then(res=>res.text())
.then(alert)

}

// ADD PRODUCT

function addProduct(){

fetch("/add-product",{

method:"POST",
headers:{"Content-Type":"application/json"},

body:JSON.stringify({

product:document.getElementById("pname").value,
price:document.getElementById("pprice").value

})

})
```

```
.then(res=>res.text())

.then(alert)

}

// LOAD CUSTOMERS INTO DROPDOWN
async function loadCustomers(){

const res = await fetch("/customers") const
data = await res.json()

let select=document.getElementById("customerSelect")

if(!select) return

select.innerHTML='<option value="">Select Customer</option>'

data.forEach(c=>{

select.innerHTML+=`<option value="${c.id}">${c.name}</option>`

})
```

```
}

// LOAD PRODUCTS INTO DROPDOWN

async function loadProducts(){

const res = await fetch("/products")
const data = await res.json()

let select=document.getElementById("productSelect")

if(!select) return

select.innerHTML='<option value="">Select Product</option>'

data.forEach(p=>{

select.innerHTML+=`<option value="${p.id}">${p.product_name}</option>`

})

}
```

```
// PLACE ORDER

function placeOrder(){

fetch("/place-order",{

method:"POST", headers:{"Content-
Type":"application/json"}, body:JSON.stringify({

customer_id:document.getElementById("customerSelect").value,
product_id:document.getElementById("productSelect").value,
quantity:document.getElementById("qty").value

})

})

.then(res=>res.text())
.then(alert)

}

// ORDER HISTORY (JOIN)
```

```
async function loadOrders(){

const    res=await    fetch("/orders")
const data=await res.json()

let
html="<table><tr><th>Customer</th><th>Product</th><th>Price</th><th>Qty</th><th>Total<
/th></tr>"
data.forEach(o=>{

html+=`

<tr>

<td>${o.name}</td>
<td>${o.product_name}</td>
<td>${o.price}</td>
<td>${o.quantity}</td>
<td>${o.total}</td>

</tr>

`
```

```
})

html+="</table>"

document.getElementById("result").innerHTML=html

}


// HIGHEST ORDER (SUBQUERY)
async function highestOrder(){

const   res=await   fetch("/highest-order")
const data=await res.json()

document.getElementById("result").innerHTML=

`<h2>Highest Order</h2>

${data[0].name} - ₹${data[0].total}`

}


// MOST ACTIVE CUSTOMER
```

```
async function activeCustomer(){

const     res=await     fetch("/active-customer")
const data=await res.json()

document.getElementById("result").innerHTML=

`<h2>Most Active Customer</h2> ${data[0].name} (${data[0].total_orders} orders)`

}



// AUTO LOAD DROPDOWNS WHEN PAGE LOADS

window.onload=function(){

loadCustomers()
loadProducts()

}
```

SERVER.JS:

```
const          express=require("express")
const db=require("./db")
```

```
const app=express()

app.use(express.json())
app.use(express.static(_dirname))

// LOGIN

app.post("/login",(req,res)=>{
const {username,password}=req.body

const sql="SELECT * FROM users WHERE username=? AND password=?"

db.query(sql,[username,password],(err,result)=>{

if(err) throw err

if(result.length>0){
res.json({success:true})
}else{
res.json({success:false})
}

})
```

```
})

// ADD CUSTOMER

app.post("/add-customer",(req,res)=>{

const {name,email}=req.body
const sql="INSERT INTO customers(name,email) VALUES(?,?)"

db.query(sql,[name,email],(err)=>{

if(err) throw err

res.send("Customer Added Successfully")

})

})

// ADD PRODUCT

app.post("/add-product",(req,res)=>{
```

```
const {product,price}=req.body

const sql="INSERT INTO products(product_name,price) VALUES(?,?)"

db.query(sql,[product,price],(err)=>{

if(err) throw err

res.send("Product Added Successfully")
})

})



// GET CUSTOMERS (for dropdown)

app.get("/customers",(req,res)=>{

const sql="SELECT * FROM customers"

db.query(sql,(err,result)=>{

if(err) throw err

res.json(result)
```

```
})

})

// GET PRODUCTS (for dropdown)

app.get("/products",(req,res)=>{
const sql="SELECT * FROM products"

db.query(sql,(err,result)=>{

if(err) throw err

res.json(result)

})

})

// PLACE ORDER

app.post("/place-order",(req,res)=>{

const {customer_id,product_id,quantity}=req.body
```

```
const sql="INSERT INTO orders(customer_id,product_id,quantity) VALUES(?,?,?)"

db.query(sql,[customer_id,product_id,quantity],(err)=>{

if(err) throw err

res.send("Order Placed Successfully")
})


})
// ORDER HISTORY (JOIN)

app.get("/orders",(req,res)=>{

const sql=`

SELECT customers.name,
products.product_name,
products.price,
orders.quantity,
(products.price * orders.quantity) AS total

FROM orders
```

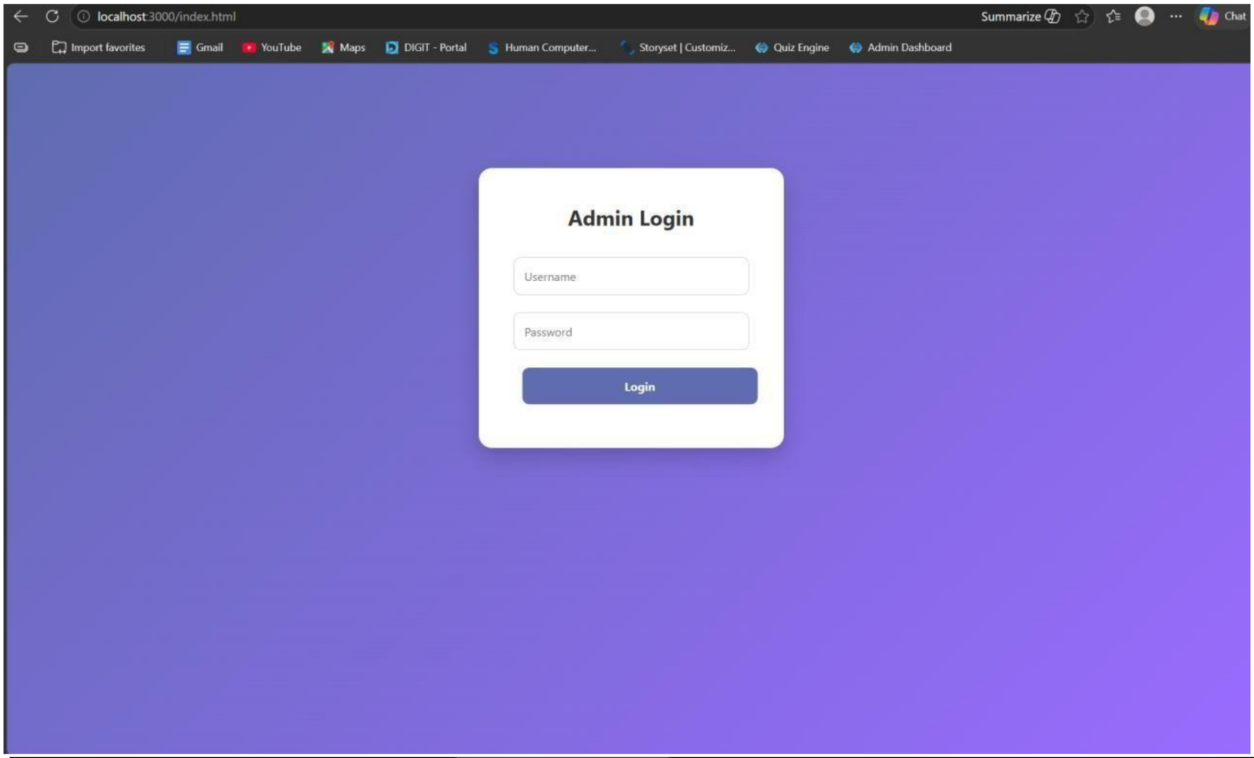
```
JOIN customers

ON orders.customer_id = customers.id


JOIN products

ON orders.product_id = products.id


ORDER BY orders.order_date DESC


`

db.query(sql,(err,result)=>{


if(err) throw err


res.json(result)


})


})


// HIGHEST VALUE ORDER (SUBQUERY)


app.get("/highest-order",(req,res)=>{


const sql=`
```

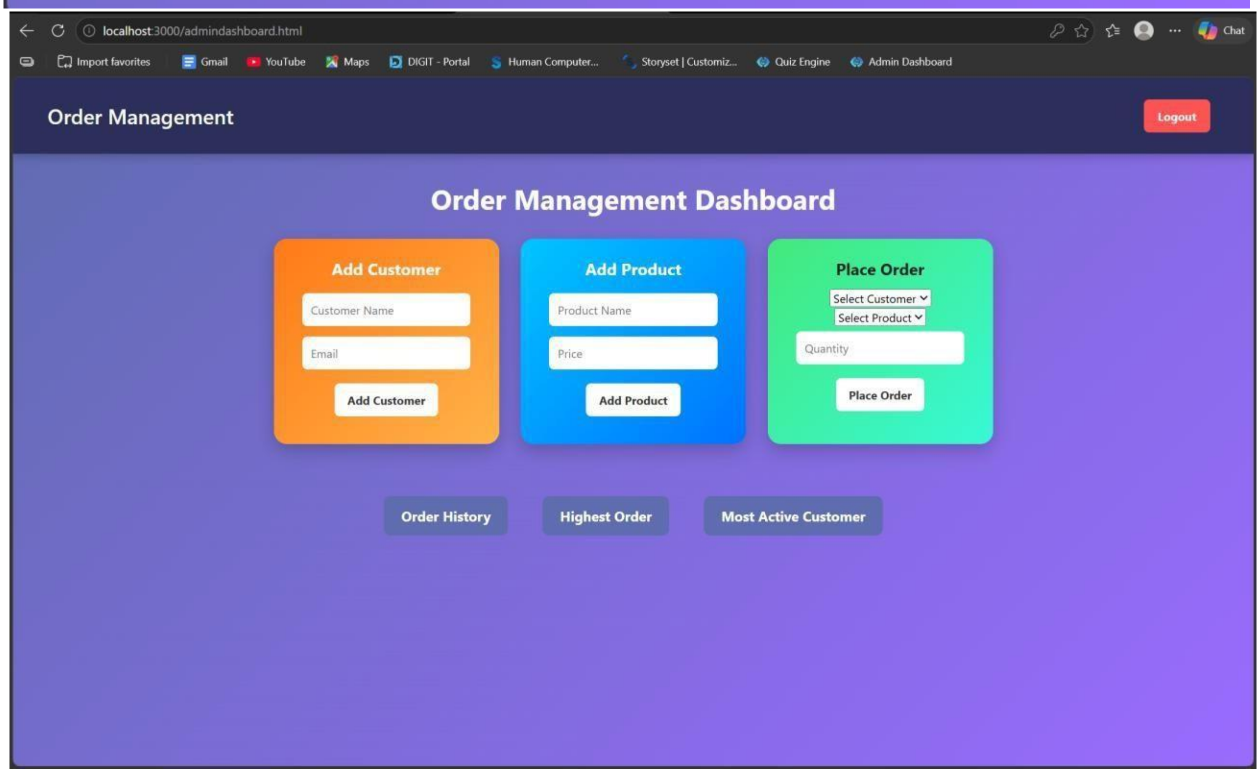
```
SELECT customers.name,
(products.price * orders.quantity) AS total

FROM orders

JOIN customers
ON orders.customer_id = customers.id

JOIN products
ON orders.product_id = products.id WHERE (products.price * orders.quantity) =

(
SELECT MAX(products.price * orders.quantity)
FROM orders
JOIN products
ON orders.product_id = products.id
)

`

db.query(sql,(err,result)=>{

if(err) throw err
```

```
res.json(result)

})

})


// MOST ACTIVE CUSTOMER

app.get("/active-customer",(req,res)=>{
const sql=`

SELECT customers.name,
COUNT(orders.id) AS total_orders

FROM customers

JOIN orders
ON customers.id = orders.customer_id

GROUP BY customers.id

ORDER BY total_orders DESC

LIMIT 1
```

```
`

db.query(sql,(err,result)=>{

if(err) throw err

res.json(result)

})
})

// START SERVER

app.listen(3000,()=>{

console.log("Server running at http://localhost:3000")

})
```

OUTPUTS:





RESULT: The system successfully retrieves and analyzes customer order data using joins and subqueries, displaying meaningful insights like highest order value and most active customer.

TASK5: Transaction-Based Payment Simulation

AIM: To simulate an online payment system using database transactions that ensures safe and reliable fund transfer between user and merchant accounts using COMMIT and ROLLBACK operations.

ALGORITHM:

1. Start the process
2. Create database tables:

    ○ User Account (UserID, Balance) ○ Merchant

    Account (MerchantID, Balance) 3. Insert initial

    balance values for user and merchant

➤ Transaction Process:

4. Begin transaction (START TRANSACTION)
5. Deduct amount from user account
6. Add amount to merchant account

➤ Validation:

7. Check if user has sufficient balance ○ If yes → proceed ○ If no → go to rollback

➤ Commit / Rollback:

8. If all operations are successful → COMMIT transaction
9. If any error occurs → ROLLBACK transaction
10. Display transaction status (Success / Failed)
11. Stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Payment Simulation</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h2>Online Payment</h2>
<select id="user" onchange="updateBalances()"></select>
<select id="merchant" onchange="updateBalances()"></select>
<input type="number" id="amount" placeholder="Enter amount">
<button onclick="pay()">Pay</button>
<h3 id="userBalance"></h3>
<h3 id="merchantBalance"></h3>
<p id="result"></p>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS: body{

```
font-family:
Arial;
background:
linear-
```

```
gradient(135deg,
#dfe9f3,#ffffff);
display:flex;
justify-
content:center;
align-
items:center;
height:100vh;
}

.container{ background:white;
padding:30px; border-radius:15px; box-
shadow:0 10px 25px rgba(0,0,0,0.2);
text-align:center; width:400px;
}

select, input{
padding:10px;
margin:10px;
width:90%; border-
radius:5px;
border:1px solid #ccc;
}
```

```
button{
padding:10px;
width:95%;
background:green;
color:white;
border:none; border-
radius:5px;
cursor:pointer; font-
size:16px;
}

button:hover{
background:darkgreen;
}

h3{
margin:5px;
color:#333;
}
```

SCRIPT.JS: let

```
usersData = [] let
merchantsData = []
function loadData(){
fetch("/balances")
```

```
.then(res=>res.json())
.then(data=>{
usersData =
data.users
merchantsData =
data.merchants let
userSelect =
document.getElemen
tById("user") let
merchantSelect =
document.getElemen
tById("merchant")
userSelect.innerHTML
=""
merchantSelect.inner
HTML=""
// fill users
usersData.forEach(u=>{ userSelect.innerHTML += `<option
value="${u.id}">${u.name}</option>`
})
window.onload = loadData // fill merchants merchantsData.forEach(m=>{
merchantSelect.innerHTML += `<option
value="${m.id}">${m.name}</option>`
})
// show initial balances
updateBalances()
```

```
})
}
function updateBalances(){ let userId =
document.getElementById("user").value let merchantId =
document.getElementById("merchant").value let user =
usersData.find(u=>u.id == userId) let merchant =
merchantsData.find(m=>m.id == merchantId)
document.getElementById("userBalance").innerText =
"User Balance: ₹" + user.balance
document.getElementById("merchantBalance").innerText = "Shop Balance: ₹" +
merchant.balance

}
function pay(){ let userId =
document.getElementById("user").value let merchantId =
document.getElementById("merchant").value let amount =
document.getElementById("amount").value fetch("/pay",{
method:"POST", headers:{
"Content-Type":"application/json"
},
body:JSON.stringify({
userId:userId,
merchantId:merchantId,
amount:amount
})
})
```

```
.then(res=>res.json()) .then(data=>{
document.getElementById("result").innerText =
data.message loadData()
})
}
window.onload = loadData
```

SERVER.JS: const express =

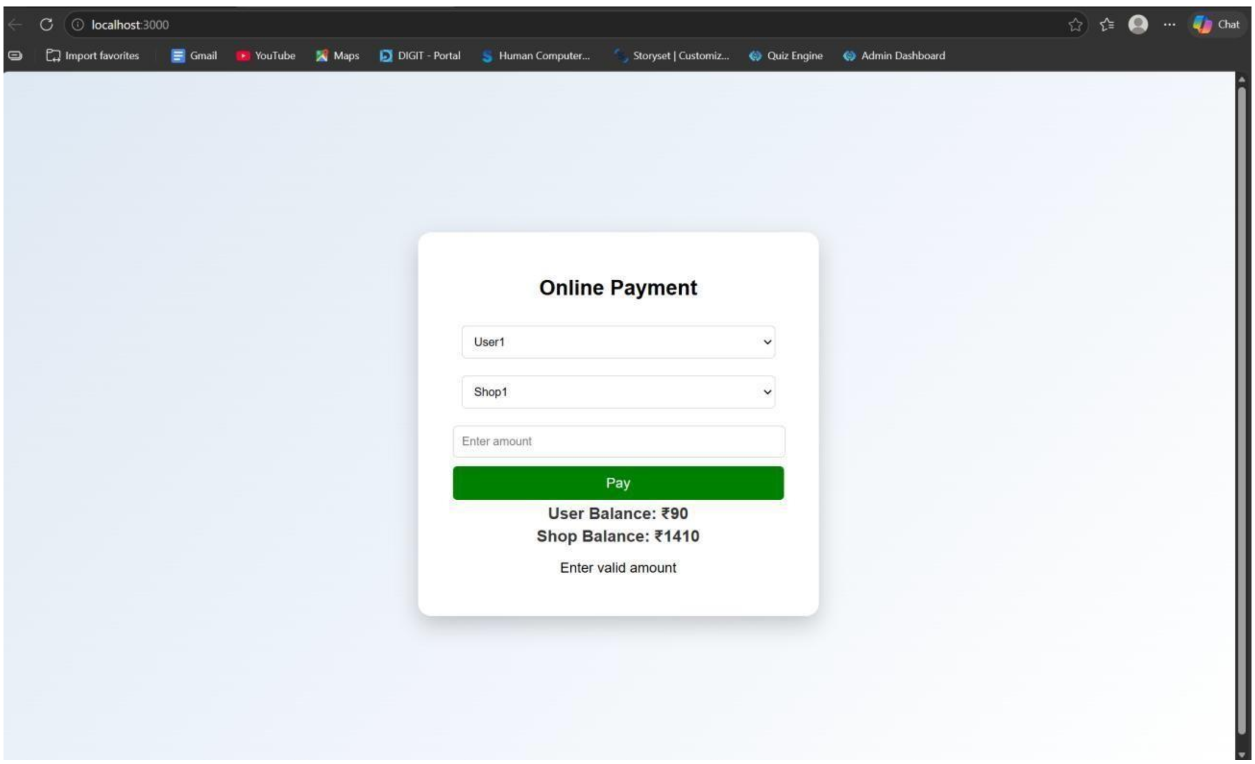
```
require("express") const db =
require("./db") const app = express()
app.use(express.json())
app.use(express.static(_dirname))
app.get("/", (req, res) => { res.sendFile(
dirname + "/index.html")
})
app.post("/pay", (req, res) => { const { userId,
  merchantId, amount } = req.body const amt =
  parseInt(amount) if (!amt || amt <= 0) { return
  res.json({ message: "Enter valid amount" })
  }
  db.beginTransaction(err => { if (err) return
    res.status(500).send("Transaction error")
    // 1. Get user balance db.query("SELECT balance FROM users WHERE id=?",
    [userId], (err, result) => { if (err || result.length === 0) { return db.rollback(() =>
    res.send("User not found"))
      }
```

```
let balance = result[0].balance if (balance < amt)
{ return db.rollback(() => { res.json({ message:
"Insufficient Balance" })
  })
}

// 2. Deduct from user db.query(
  "UPDATE users SET balance = balance - ? WHERE id=?",
  [amt, userId],
  (err) => {
if (err) {
      return db.rollback(() => res.send("Deduction failed"))
    }

    // 3. Add to merchant db.query(
      "UPDATE merchants SET balance = balance + ? WHERE id=?",
      [amt, merchantId],
      (err) => {
if (err) {
          return db.rollback(() => res.send("Merchant update failed"))
        }
        // 4. Commit
        db.commit(err => {
```

```
                        if (err) {
                          return db.rollback(() => res.send("Commit failed"))
                        }

                        res.json({ message: "Payment Successful    " })
                      })
                    }
                  )
                }
              )
            })
          })
        })
        app.get("/balances", (req, res) => {

          db.query("SELECT * FROM users", (err, users) => {

            if (err) return res.send("Error fetching users")

            db.query("SELECT * FROM merchants", (err, merchants) => {

              if (err) return res.send("Error fetching merchants")

              res.json({   users:   users,
                merchants: merchants
```

```
        })
      })
    })
})
app.listen(3000, () => { console.log("Server running on
  http://localhost:3000")
})
```

OUTPUTS:



RESULT: The system successfully performs secure payment transactions by updating balances accurately using COMMIT on success and ROLLBACK on failure.

TASK6: Automated Logging using Triggers & Views

AIM: To develop a web-based application for automated logging of database activities using MySQL triggers and views, integrated with a Node.js backend and frontend interface, in order to track INSERT and UPDATE operations and generate daily activity reports.

ALGORITHM:

1. Start the program
2. Create database and required tables
3. Create triggers for INSERT and UPDATE operations
4. Store actions in log table automatically
5. Create a view for daily activity report
6. Develop backend using Node.js and Express
7. Connect backend with MySQL database
8. Create APIs for add, update, logs, and report
9. Design frontend using HTML, CSS, and JavaScript
10. Run the application and display results
11. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

   <title>Employee Audit System</title>

   <link rel="stylesheet" href="style.css">

</head>

<body>

<h2>Employee Management</h2>

<div class="form">
```

```
   <input type="number" id="emp_id" placeholder="Employee ID">
   <input type="text" id="name" placeholder="Name">
   <input type="number" id="salary" placeholder="Salary">
   <button onclick="addEmployee()">Add</button>
</div>
<div class="form">
   <input type="number" id="update_id" placeholder="Employee ID">
   <input type="number" id="new_salary" placeholder="New Salary">
   <button onclick="updateEmployee()">Update</button>
</div>
<button onclick="loadLogs()">View Logs</button>
<button onclick="loadReport()">View Daily Report</button>
<h3>Output</h3>
<pre id="output"></pre>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-
family:     Arial;     text-
align:                 center;
background: #f4f4f4;
}
.form  {  margin:
   15px;
}
```

```
input {
   padding: 8px;
   margin: 5px;
}
button { padding:
   8px 15px;
   background: blue;
   color: white;
   border: none;
   cursor: pointer;
}
button:hover { background:
   darkblue;
}
pre { background:
   white; padding:
   10px; width: 60%;
   margin: auto;
   text-align: left;
}
```

SCRIPT.JS:

```
const BASE_URL = "http://localhost:3000"; // Add Employee function addEmployee() { const data
= { emp_id: document.getElementById("emp_id").value, name:
document.getElementById("name").value, salary: document.getElementById("salary").value
   };
```

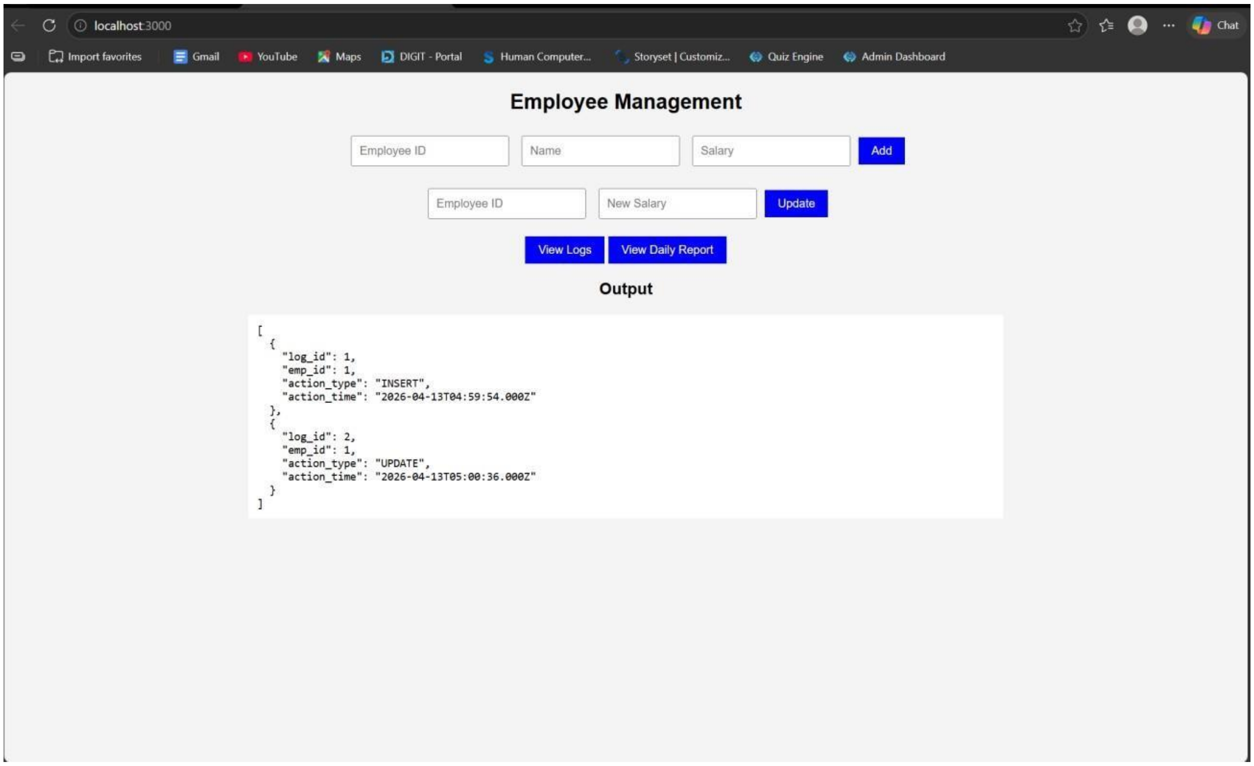
```
  fetch(`${BASE_URL}/add`,  {  method:  "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  })
  .then(res => res.text())
  .then(data => alert(data));
}
// Update Employee function updateEmployee() { const id =
document.getElementById("update_id").value; const salary =
document.getElementById("new_salary").value;

  fetch(`${BASE_URL}/update/${id}`,  {  method:
    "PUT",    headers:    {    "Content-Type":
    "application/json"  },  body:  JSON.stringify({
    salary })
  })
  .then(res => res.text())
  .then(data => alert(data));
}
//   Load   Logs   function
loadLogs()          {
fetch(`${BASE_URL}/logs`)
  .then(res => res.json())
  .then(data               =>               {
    document.getElementById("output").innerText =
      JSON.stringify(data, null, 2);
```

```
  });
}
// Load Report function
loadReport() {
fetch(`${BASE_URL}/report`)
  .then(res => res.json())
  .then(data => {
    document.getElementById("output").innerText =
      JSON.stringify(data, null, 2);
  });
}
```

DB.JS:

```
const mysql = require('mysql2');
const db = mysql.createConnection({
host: 'localhost', user: 'root',
password: 'fahi',
  database: 'audit_db'
});
db.connect((err) => {
  if (err) {
    console.error('DB Connection Failed:', err);
  } else {
    console.log('Connected to MySQL');
  }
});
module.exports = db;
```

OUTPUTS:



RESULT: The system successfully logs all INSERT and UPDATE operations automatically using triggers and displays daily activity reports through a web interface.

TASK7: Interactive Web Form with Events & Functions

AIM: To develop an interactive web-based feedback form with real-time validation, UI effects, and backend data storage using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design feedback form with input fields (name, email, message)
4. Apply CSS styling for layout and visual effects
5. Define reusable validation functions in JavaScript

6. Capture user input using DOM elements
7. Apply keyup event for real-time validation
8. Add hover effect to highlight input fields
9. Implement double-click event on submit button
10. Validate all fields before submission
11. Send data to server using fetch API (POST request)
12. Store data in database module and display success message

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <title>Feedback Form</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

  <h2>   Customer Feedback</h2>

  <form id="feedbackForm">

    <input type="text" id="name" placeholder="Your Name" required>

    <span class="error" id="nameError"></span>

    <input type="email" id="email" placeholder="Your Email" required>

    <span class="error" id="emailError"></span>

    <textarea id="message" placeholder="Your Feedback" required></textarea>
```

```
        <span class="error" id="msgError"></span>
        <button type="button" id="submitBtn">Submit</button>
    </form>
    <p id="successMsg"></p>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-family: 'Segoe UI', sans-
serif; background: linear-gradient(135deg, #1f1c2c,
#928dab); display: flex; justify-content: center; align-items:
center; height: 100vh; color: white;

}
.container { background: rgba(255, 255,
    255, 0.1); padding: 25px; border-radius:
    15px; backdrop-filter: blur(10px); width:
    320px; box-shadow: 0 10px 25px
    rgba(0,0,0,0.3);
}
h2 { text-align:
    center;
}
input, textarea {
    width: 100%;
    padding: 10px;
    margin: 8px 0;
    border-radius:
```

```
    8px; border: none;
    outline:        none;
    transition: 0.3s;
}
input:hover,   textarea:hover   {
    transform:  scale(1.05);  box-
    shadow: 0 0 8px #00f7ff;
}
button {
    width:            100%;
    padding:          10px;
    background:
    #00c6ff;       border:
    none; border-radius:
    10px;   color:   white;
    font-weight:     bold;
    cursor: pointer;
}
button:hover { background:
    #0072ff;
}
.error    {    color:
    #ff6b6b;   font-
    size: 12px;
}
```

```
#successMsg { text-
   align:        center;
   margin-top: 10px;
   color: #00ffae;
}
```

SCRIPT.JS:

```
const nameInput = document.getElementById("name"); const
emailInput    =    document.getElementById("email");    const
messageInput     =     document.getElementById("message");
const  submitBtn  =  document.getElementById("submitBtn");
function  validateName()  {  if  (nameInput.value.length  <  3)  {
document.getElementById("nameError").innerText  =  "Min  3
characters"; return false;
   }
   document.getElementById("nameError").innerText      =      "";
   return true;
}
function  validateEmail()  {  let  pattern  =  /^[^  ]+@[^  ]+\.[a-z]{2,3}$/;  if
   (!emailInput.value.match(pattern))                                        {
   document.getElementById("emailError").innerText  =  "Invalid  email";
   return false;
   }
   document.getElementById("emailError").innerText      =      "";
   return true;
}
```

```
function validateMessage() { if (messageInput.value.length < 5) {
  document.getElementById("msgError").innerText = "Too short";
  return false;
  }
  document.getElementById("msgError").innerText    =    "";
  return true;
}
nameInput.addEventListener("keyup", validateName); emailInput.addEventListener("keyup",
validateEmail); messageInput.addEventListener("keyup", validateMessage);
submitBtn.addEventListener("dblclick", () => { if (validateName() && validateEmail() &&
validateMessage()) { sendData();
  }   else   {   alert("Fix   errors   before
    submitting!");
  }
});
function sendData() {
  fetch("http://localhost:3000/submit",
  { method: "POST", headers: {
      "Content-Type": "application/json"
    },
    body:   JSON.stringify({   name:
      nameInput.value,        email:
      emailInput.value,    message:
      messageInput.value
    })
  })
```

```
  .then(res => res.json())
  .then(data => { document.getElementById("successMsg").innerText = "   Feedback
    Submitted!";
  });
}
```

SERVER.JS: const express = require("express");

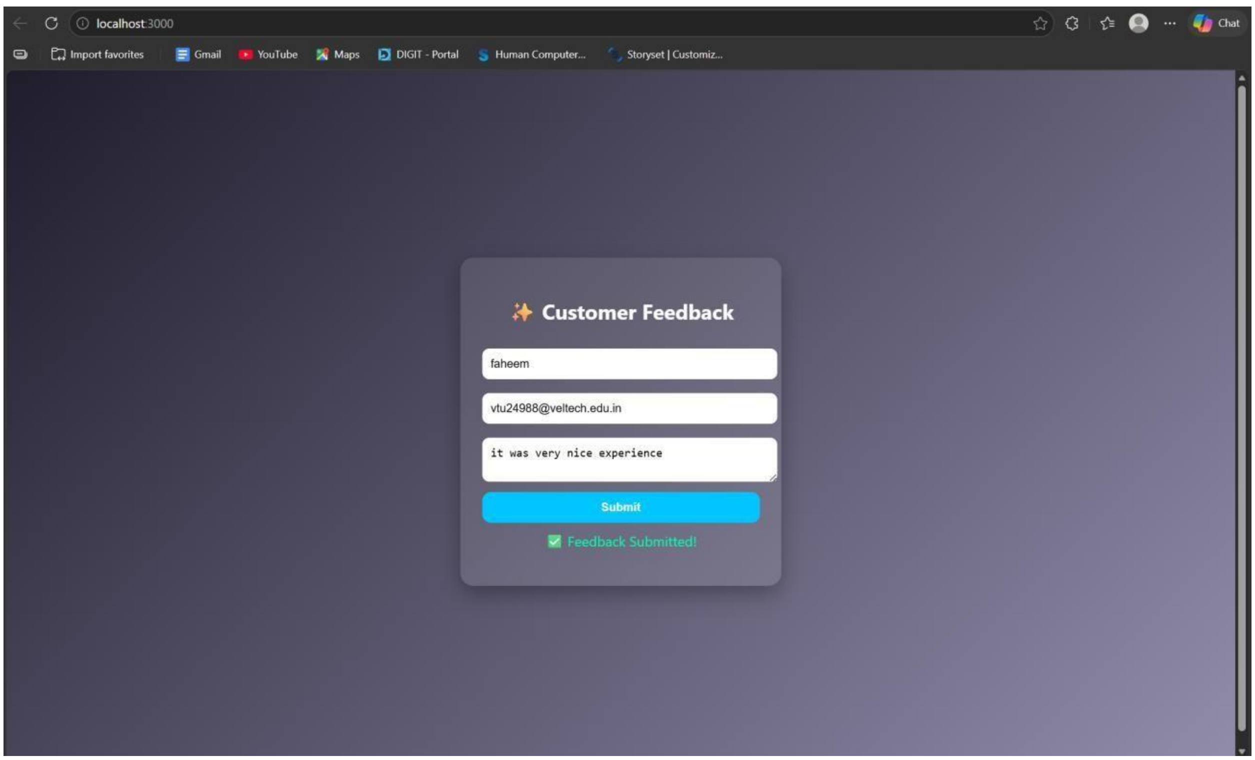
```
const cors = require("cors"); const bodyParser =
require("body-parser"); const path =
require("path"); const db = require("./db"); const
app = express(); app.use(cors());
app.use(bodyParser.json()); app.use(express.static(
dirname)); app.get("/", (req, res) => {
res.sendFile(path.join(_dirname, "index.html"));
});
app.post("/submit", (req, res) => { const {
  name, email, message } = req.body;
  db.saveFeedback({ name, email, message });
  res.json({ status: "success" });
});
app.listen(3000, () => { console.log("Server running on
  http://localhost:3000");
});
```

DB.JS:

```
let feedbacks = []; function
saveFeedback(data) {
```

```
feedbacks.push(data);

console.log("Saved:", data);

}

module.exports = {
  saveFeedback
};
```

OUTPUT:



RESULT: The interactive feedback form was successfully implemented with real-time validation, enhanced UI effects, and backend data handling.

TASK8: Employee Management

AIM: To develop a simple Employee Management system using HTML, CSS, JavaScript, and Node.js for adding and displaying employee data.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design the user interface with input fields (name, role)
4. Apply CSS styling for layout and centering
5. Capture user input using JavaScript
6. Create function to add employee data
7. Send data to server using fetch API (POST request)
8. Create backend using Node.js and Express
9. Receive and store employee data in memory (db.js)
10. Create API to fetch all employee records (GET request)
11. Display employee list dynamically on the webpage
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
   <title>Employee Management</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
   <h2>Employee Management</h2>
```

```
   <input id="name" placeholder="Name">
   <input id="role" placeholder="Role">
   <button onclick="addEmployee()">Add</button>
   <ul id="list"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-
family: Arial;
background: #1e1e2f;
color: white; display:
flex; justify-content:
center; align-items:
center; height: 100vh;
margin: 0;
}
.container { text-align:
   center;
}
input { padding:
   10px; margin: 5px;
   border-radius:
   5px; border: none;
}
```

```
button { padding: 10px;
   background:
   #00c6ff;       border:
   none;  border-radius:
   5px; cursor: pointer;
}
```

SCRIPT.JS:

```
function      addEmployee()      {      const      name      =
   document.getElementById("name").value;  const  role  =
   document.getElementById("role").value;
   fetch("http://localhost:3000/add",  {  method:  "POST",
   headers: {
         "Content-Type": "application/json"
      },
      body: JSON.stringify({ name, role })
   })
   .then(res => res.json())
   .then(() => loadEmployees());
}
function loadEmployees() { fetch("http://localhost:3000/all")
   .then(res => res.json())
   .then(data      =>      {      let      list      =
      document.getElementById("list");
      list.innerHTML = "";
```

```
    data.forEach(emp    =>    {    let    li    =
      document.createElement("li");
      li.innerText = emp.name + " - " + emp.role;
      list.appendChild(li);
    });
  });
}
loadEmployees();
```

SERVER.JS:

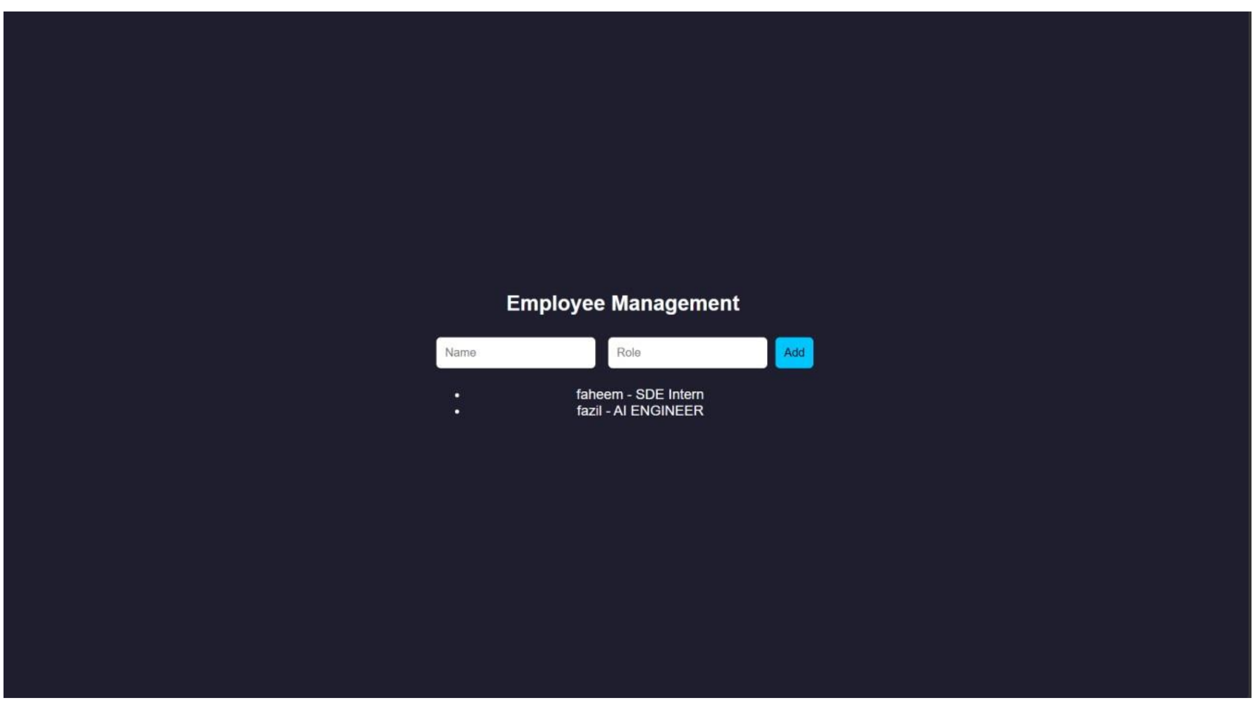
```
const express = require("express");
const cors = require("cors"); const
path = require("path"); const db =
require("./db"); const app =
express(); app.use(cors());
app.use(express.json());
app.use(express.static(_dirname));
app.get("/", (req, res) => {
res.sendFile(path.join(_dirname,
"index.html"));
});
app.post("/add", (req, res) => {
  db.addEmployee(req.body);
  res.json({ msg: "Added" });
});
app.get("/all", (req, res) => { res.json(db.getEmployees());
```

```
});
app.listen(3000, () => { console.log("•    Server running at:
   http://localhost:3000");
});
```

DB.JS:

```
let employees = []; function
addEmployee(emp)          {
employees.push(emp);
}
function getEmployees() { return
   employees;
}
module.exports = { addEmployee, getEmployees };
```

OUTPUT:



RESULT: The Employee Management system was successfully implemented with a userfriendly interface and dynamic data handling using Node.js backend.

TASK9: Employee MVC APP

AIM: To develop a basic MVC-based Employee application using HTML, CSS, JavaScript, and Node.js to handle user requests and display employee details.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI form to accept employee details
4. Apply CSS styling for better user interface
5. Capture user input using JavaScript
6. Send request to server using fetch API
7. Create controller logic in server.js
8. Define routes to handle requests (/add, /all)
9. Store employee data in memory using db.js
10. Retrieve employee data from database module
11. Send response back to frontend (view)
12. Display employee details dynamically and stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>
```

```html
    <title>Employee MVC</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
  <div class="container">
    <h2>Employee Details</h2>
    <input id="name" placeholder="Name">
    <input id="role" placeholder="Role">
    <button onclick="addEmployee()">Add</button>
    <ul id="list"></ul>
  </div>
  <script src="script.js"></script>
  </body>
  </html>
```

STYLE.CSS:

```css
* {
  margin: 0; padding: 0;
  box-sizing:    border-
  box;
}
body { font-family: 'Segoe UI', sans-serif; background:
  linear-gradient(135deg, #1f1c2c, #928dab); height:
  100vh;
```

```
    display: flex; justify-
    content: center; align-
    items: center;

    color: white;
}
.container { background: rgba(255, 255,
    255, 0.1); padding: 30px; border-
    radius: 15px; backdrop-filter:
    blur(10px); width: 320px;
    text-align: center;

    box-shadow: 0 10px 25px rgba(0,0,0,0.4);
}
h2 { margin-bottom:
    15px;
}
input { width:
    100%;
    padding: 10px;
    margin: 8px 0;

    border: none;
    border-radius:
    8px; outline: none;
```

```
  transition: 0.3s;
}
input:hover,
input:focus   {   transform:
  scale(1.05); box-shadow: 0 0
  8px #00f7ff;
}
button { width:
  100%;
  padding: 10px;
  margin-top: 10px;

  background:       #00c6ff;
  border: none;
  border-radius: 10px;

  color: white; font-
  weight:    bold;
  cursor: pointer;

  transition: 0.3s;
}
button:hover       {
  background: #0072ff;
```

```
transform:
scale(1.05);
} ul { list-style: none;
margin-top: 15px;

} li
{
background:
rgba(255,255,255,0.15); padding:
8px; margin: 5px 0;
border-radius: 6px;


transition: 0.3s;
}
li:hover { background:
rgba(255,255,255,0.3);
}
```

SCRIPT.JS:

```
function addEmployee() { const name =
document.getElementById("name").value; const role =
document.getElementById("role").value; fetch("/add", {
method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify({ name, role })
})
.then(() => loadEmployees());
}
```

```
function loadEmployees() { fetch("/all")
  .then(res => res.json())
  .then(data   =>   {   let   list   =
    document.getElementById("list");
    list.innerHTML = ""; data.forEach(emp => {
    let  li  =  document.createElement("li");
    li.innerText = emp.name + " - " + emp.role;
    list.appendChild(li);
    });
  });
}
loadEmployees();
```

SERVER.JS:

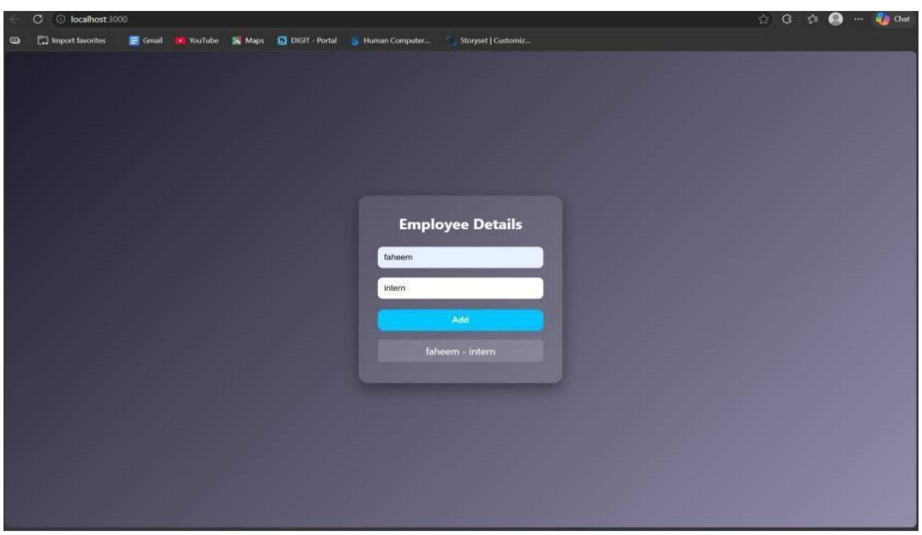
```
const  express  =  require("express");
const cors = require("cors"); const path
=  require("path");  const  db  =
require("./db"); const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(_____dirname));
app.post("/add",  (req,  res)  =>  {
db.addEmployee(req.body);  res.json({
msg: "Employee Added" });
});
app.get("/all", (req, res) => { res.json(db.getEmployees());
```

```
});
app.listen(3000, () => {
  console.log("http://localhost:3000");
});
```

DB.JS:

```
let employees = []; function
addEmployee(emp) {
employees.push(emp);
}
function getEmployees() { return
  employees;
}
module.exports = { addEmployee, getEmployees };
```

OUTPUT:



RESULT: The MVC-based Employee application was successfully implemented using Node.js, where user requests were processed and employee details were displayed dynamically.

TASK10: Student Crud App

AIM: To develop a Student CRUD application using HTML, CSS, JavaScript, and Node.js to perform create, read, update, and delete operations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for student input (id, name, course)
4. Apply CSS for styling and layout
5. Capture user input using JavaScript
6. Send data to server using fetch API
7. Create backend using Node.js and Express
8. Implement Create operation (POST request)
9. Implement Read operation (GET request)
10. Implement Update operation (PUT request)
11. Implement Delete operation (DELETE request)
12. Display updated student data dynamically and stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>Student CRUD</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
    <h2>Student CRUD</h2>
    <input id="id" placeholder="ID">
    <input id="name" placeholder="Name">
    <input id="course" placeholder="Course">
    <button onclick="addStudent()">Add</button>
    <button onclick="updateStudent()">Update</button>
    <ul id="list"></ul>
</div>
<script src="script.js"></script>
</body>
</html>  STYLE.CSS:  body  {  font-family:  'Segoe  UI';
background:   linear-gradient(135deg,#1f1c2c,#928dab);
height: 100vh; display:flex; justify-content:center; align-
items:center; color:white;
}
.container          {          background:
    rgba(255,255,255,0.1);
    padding:20px;   border-radius:10px;
    text-align:center;

}
input                    {
    display:block;
    margin:8px
    auto;
    padding:8px;
}
```

```
button { margin:5px;
   padding:8px;
   background:#00c6ff
   ; border:none;
   color:white;
}
```

SCRIPT.JS:

```
function addStudent() { let data = getInput();
   fetch("/add", { method: "POST", headers:
   {"Content-Type":"application/json"}, body:
   JSON.stringify(data)
   }).then(loadStudents);
}
function loadStudents() {
   fetch("/all")
   .then(res => res.json())
   .then(data => {
      let list = document.getElementById("list");
      list.innerHTML = ""; data.forEach(s => { let
      li = document.createElement("li");
      li.innerHTML = `
            ${s.id} - ${s.name} (${s.course})
              <button onclick="deleteStudent(${s.id})">  </button>
         `;
         list.appendChild(li);
      });
```

```
  });
}
function updateStudent() { let data = getInput();
  fetch("/update", { method: "PUT", headers:
  {"Content-Type":"application/json"},        body:
  JSON.stringify(data)
  }).then(loadStudents);
}
function deleteStudent(id) { fetch(`/delete/${id}`,
  { method: "DELETE" })
  .then(loadStudents);
}
function getInput() { return
  {
    id: document.getElementById("id").value, name:
    document.getElementById("name").value,
    course:
    document.getElementById("course").value
  };
}
loadStudents();
```

SERVER.JS:

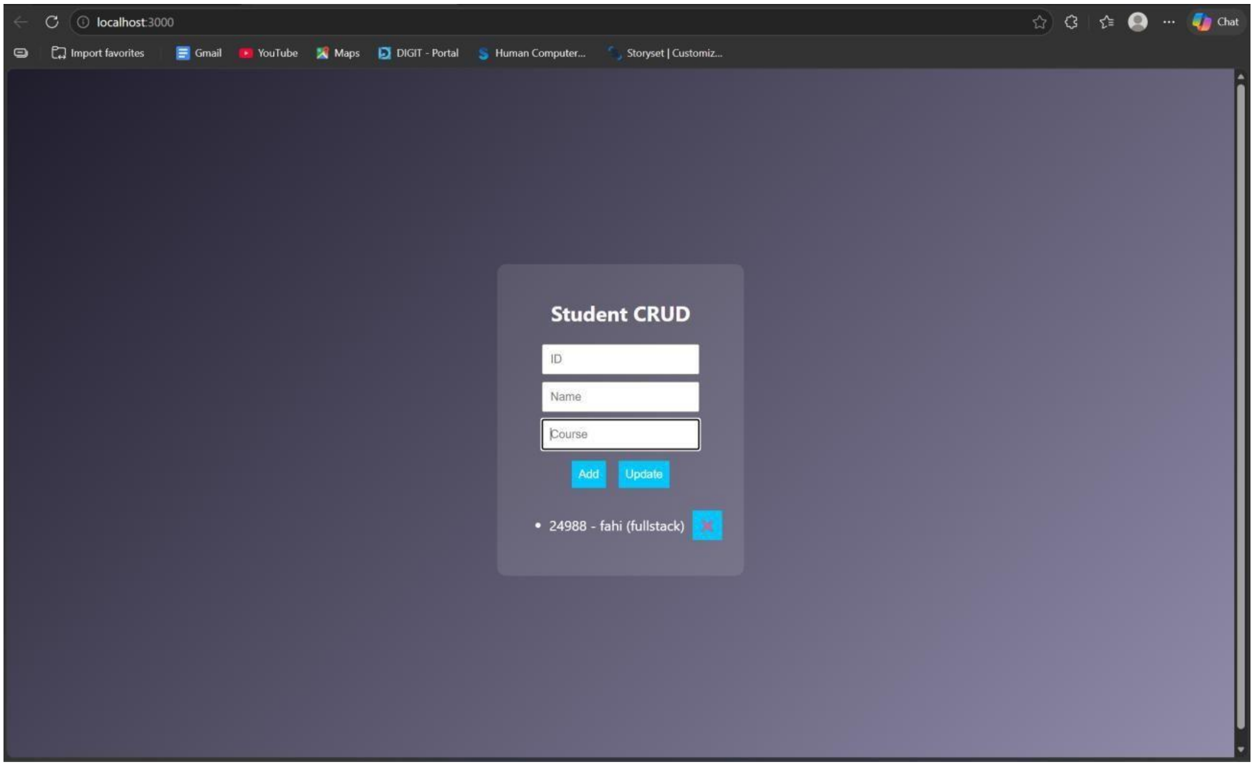
```
const express = require("express");
const cors = require("cors"); const
db = require("./db"); const app =
```

```
express(); app.use(cors());
app.use(express.json());
app.use(express.static(_dirname));
// CREATE app.post("/add",
(req, res) => {
db.add(req.body);
res.json({msg:"Added"});
});
// READ app.get("/all", (req,
res) => {
res.json(db.getAll());
});
// UPDATE app.put("/update",
(req, res) => {
db.update(req.body);
res.json({msg:"Updated"});
});
// DELETE app.delete("/delete/:id",
(req, res) => {
db.remove(req.params.id);
res.json({msg:"Deleted"});
});
app.listen(3000, () => console.log("http://localhost:3000"));
```

DB.JS:

```
let students = [];
function add(s) {
students.push(s);
}
function getAll() { return
   students;
}
function update(updated) { students
   = students.map(s =>
      s.id == updated.id ? updated : s
   );
}
function remove(id) { students =
   students.filter(s => s.id != id);
}
module.exports = { add, getAll, update, remove };
```

OUTPUT:



RESULT: The Student CRUD application was successfully implemented with full create, read, update, and delete operations using Node.js and a dynamic web interface.

TASK11: Student Data Layer

AIM: To implement a data access layer for managing student records with filtering, sorting, and pagination using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for student details (id, name, age, department)
4. Apply CSS styling for layout and UI
5. Capture input using JavaScript
6. Send data to backend using fetch API
7. Create Node.js server using Express

8. Store student data in memory (db.js)

9. Implement filtering (by department/age)

10. Implement sorting (by name/age)

11. Implement pagination (limit & page number)

12. Display filtered, sorted, paginated data

PROGRAM:

INDEX.HTML:

```html
<!DOCTYPE html>

<html>

<head>

  <title>Student Data Layer</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

  <h2>Student Data</h2>

  <input id="name" placeholder="Name">

  <input id="age" placeholder="Age">

  <input id="dept" placeholder="Department">

  <button onclick="addStudent()">Add</button>

  <hr>

  <input id="filterDept" placeholder="Filter Dept">

  <input id="sortBy" placeholder="Sort (name/age)">

  <input id="page" placeholder="Page No">

  <button onclick="loadStudents()">Search</button>

  <ul id="list"></ul>
```

```
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-family: Arial;
background: #1e1e2f;
color: white;
display:flex; justify-
content:center; align-
items:center;
height:100vh;
}
.container { text-
  align:center;
}
input {
  display:block;
  margin:5px;
  padding:8px;
}
```

SCRIPT.JS:

```
function addStudent() {
  const data = { name:
  val("name"), age:
  val("age"), dept:
  val("dept")
```

```
    };
    fetch("/add", { method: "POST", headers:
      {"Content-Type":"application/json"}, body:
      JSON.stringify(data)
    }).then(loadStudents);
}
function loadStudents() { const
    dept = val("filterDept"); const
    sort = val("sortBy"); const
    page = val("page");
    fetch(`/students?dept=${dep
    t}&sort=${sort}&page=${pag
    e}`)

    .then(res => res.json())
    .then(data => { let list =
      document.getElementById("list");
      list.innerHTML = ""; data.forEach(s => { let li =
      document.createElement("li"); li.innerText =
      `${s.name} (${s.age}) - ${s.dept}`;
      list.appendChild(li);
      });
    });
}
function val(id){ return
    document.getElementById(id).value;
```

}

loadStudents();

SERVER.JS: const express =

require("express"); const cors =

require("cors"); const db =

require("./db"); const app =

express(); app.use(cors());

app.use(express.json());

app.use(express.static(_dirname));

// Add student app.post("/add",

(req, res) => { db.add(req.body);

res.json({msg:"Added"});

});

// Filter + Sort + Pagination

app.get("/students", (req, res) => { let {

dept, sort, page } = req.query; let data =

db.getAll(); // FILTER if (dept) { data =

data.filter(s => s.dept === dept);

}

// SORT if (sort === "name") {

data.sort((a,b)=>a.name.localeCompare(b.name));

} else if (sort === "age") { data.sort((a,b)=>a.age

- b.age);

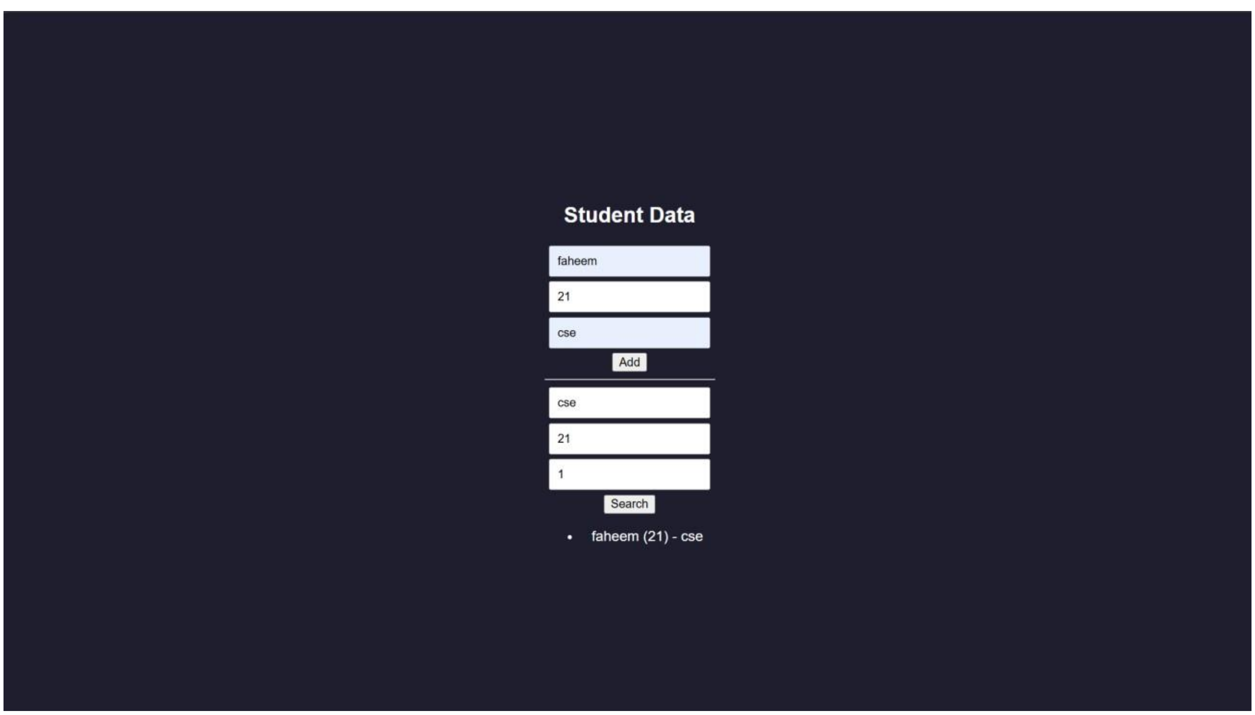
}

// PAGINATION

```
let limit = 3; page = parseInt(page) || 1; let
start = (page - 1) * limit; let paginated =
data.slice(start, start + limit);
res.json(paginated);
});
app.listen(3000, () => console.log("http://localhost:3000"));
```
DB.JS:

```
let students = [];
function add(s) {
students.push(s);
}
function getAll() { return
students;
}
module.exports = { add, getAll };
```

OUTPUT:



RESULT: The data access layer was successfully implemented with filtering, sorting, and pagination using Node.js and dynamic frontend interaction.

TASK12: Product Api

AIM: To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM:

1. Start the program
2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)
4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>Product API</title>
```

```html
</head>

<body>

<h2>Product API Test</h2>

<input id="id" placeholder="ID">

<input id="name" placeholder="Name">

<input id="price" placeholder="Price">

<button onclick="add()">Add</button>

<button onclick="get()">Get All</button>

<ul id="list"></ul>

<script src="script.js"></script>

</body>

</html>
```

STYLE.CSS:

```css
/* Reset */

* {

  margin: 0; padding: 0;

  box-sizing:       border-

  box;

}

/* Background */ body { font-family: 'Segoe UI', sans-serif;

background: linear-gradient(135deg, #141e30, #243b55);

height: 100vh; display: flex; justify-content: center; align-

items: center; color: white;

}

/* Container */
```

```
.container { background: rgba(255, 255,
   255,  0.1);  padding:  25px;  border-
   radius: 15px;
   backdrop-filter: blur(10px);


   width:  350px;  text-align:  center;  box-
   shadow: 0 10px 25px rgba(0,0,0,0.4);
}
/*  Heading  */  h2  {
margin-bottom: 15px;
}
/* Inputs */ input {
width:          100%;
padding:        10px;
margin:     8px     0;
border:          none;
border-radius:    8px;
outline:         none;
transition: 0.3s;
}
/* Hover + focus effect */ input:hover,

input:focus      {      transform:
   scale(1.05); box-shadow: 0 0
   8px #00f7ff;
}
```

```
/* Buttons */ button {
width: 100%; padding:
10px; margin-top: 8px;
background: #00c6ff;
border: none; border-
radius: 10px; color:
white; font-weight:
bold; cursor: pointer;
transition: 0.3s;
}
/* Button hover */
button:hover {
background: #0072ff;
transform: scale(1.05);
}
/* Product List */
ul { list-style: none;
  margin-top:
  15px;

}
/* List items */
li {
  background:
  rgba(255,255,255,0.15); padding:
  10px; margin: 6px 0; border-radius:
  8px; display: flex; justify-content:
```

```
space-between; align-items: center;
transition: 0.3s;
}
/* Hover effect */ li:hover {
background: rgba(255,255,255,0.3);
}
/* Optional delete button inside list */
li button { width: auto; padding: 5px
8px; font-size: 12px; background:
#ff4b5c;
}
li button:hover { background:
#ff1e38;
}
```

SCRIPT.JS: function add() { fetch("/products", {

```
method: "POST", headers: {"Content-
Type":"application/json"}, body:
JSON.stringify(getData())
}).then(get);
}
function get() { fetch("/products")
.then(res => res.json())
.then(data => { let list =
document.getElementById("list"); list.innerHTML
= ""; data.forEach(p => { let li =
document.createElement("li"); li.innerText =
```

```
    `${p.id}      -      ${p.name}      -      ₹${p.price}`;
    list.appendChild(li);
    });
  });
}
function      getData()      {      return      {      id:
  document.getElementById("id").value,        name:
  document.getElementById("name").value,    price:
  document.getElementById("price").value

  };
}
```

SERVER.JS:

```
const express = require("express");
const cors = require("cors"); const
db = require("./db"); const app =
express();           app.use(cors());
app.use(express.json()); // GET all
products      app.get("/products",
(req, res) => { res.json(db.getAll());
});
//         POST         add         product
app.post("/products",   (req,   res)   =>   {
db.add(req.body);   res.json({   message:
"Product added" });
```

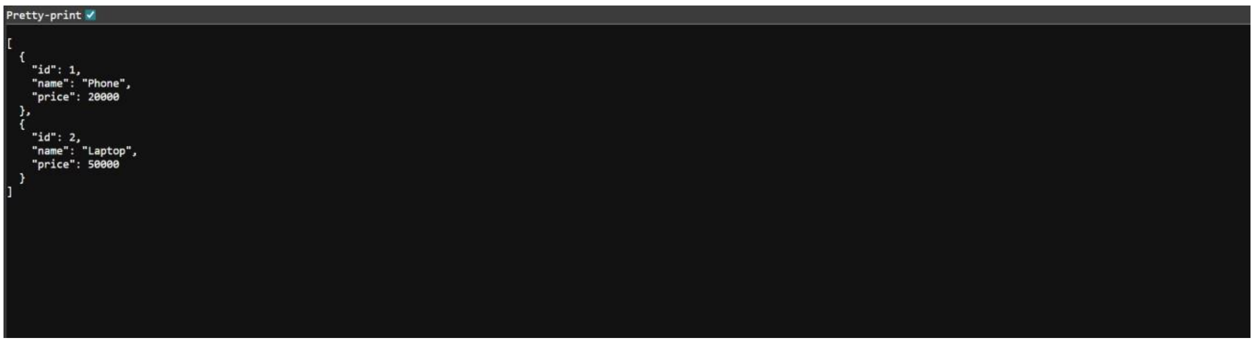
```
});
//          PUT          update          product
app.put("/products/:id",   (req,   res)   =>   {
db.update(req.params.id,           req.body);
res.json({ message: "Product updated" });
});
// DELETE product app.delete("/products/:id",
(req,   res)   =>   {   db.remove(req.params.id);
res.json({ message: "Product deleted" });
});
// Server start app.listen(3000, () => { console.log("•    API running
at http://localhost:3000/products");
});
```

DB.JS:

```
let  products  =  [];
function  getAll()  {
return products;
}
function add(p) { products.push(p);
}
function update(id, newData) { products
   = products.map(p =>
      p.id == id ? { ...p, ...newData } : p
   );
}
```

```
function     remove(id)     {     products     =
   products.filter(p => p.id != id);
}
module.exports = { getAll, add, update, remove };
```

OUTPUT:



RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK13: PRODUCT-API

AIM: To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM:

1. Start the program
2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)
4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>Product API</title>

</head>

<body>

<h2>Product API Test</h2>
```

```html
<input id="id" placeholder="ID">
<input id="name" placeholder="Name">
<input id="price" placeholder="Price">
<button onclick="add()">Add</button>
<button onclick="get()">Get All</button>
<ul id="list"></ul>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```css
/* Reset */
* {
   margin: 0; padding: 0;
   box-sizing: border-
   box;
}
/* Background */ body { font-family: 'Segoe UI', sans-serif;
background: linear-gradient(135deg, #141e30, #243b55);
height: 100vh; display: flex; justify-content: center; align-
items: center; color: white;
}
/* Container */
.container { background: rgba(255, 255,
   255, 0.1); padding: 25px; border-
   radius: 15px;
   backdrop-filter: blur(10px);
```

```
width: 350px; text-align: center; box-
shadow: 0 10px 25px rgba(0,0,0,0.4);
}
/* Heading */ h2 {
margin-bottom: 15px;
}
/* Inputs */ input {
width: 100%;
padding: 10px;
margin: 8px 0;
border: none;
border-radius: 8px;
outline: none;
transition: 0.3s;
}
/* Hover + focus effect */ input:hover,
input:focus { transform:
scale(1.05); box-shadow: 0 0
8px #00f7ff;
}
/* Buttons */ button {
width: 100%; padding:
10px; margin-top: 8px;
background: #00c6ff;
```

```
border: none; border-
radius: 10px; color:
white; font-weight:
bold; cursor: pointer;
transition: 0.3s;
}
/* Button hover */
button:hover {
background: #0072ff;
transform: scale(1.05);
}
/* Product List */
ul { list-style: none;
  margin-top:
  15px;
}
/* List items */
li {
  background:
  rgba(255,255,255,0.15); padding:
  10px; margin: 6px 0; border-radius:
  8px; display: flex; justify-content:
  space-between; align-items: center;
  transition: 0.3s;
}
```

```
/* Hover effect */ li:hover {
background: rgba(255,255,255,0.3);
}
/* Optional delete button inside list */
li button { width: auto; padding: 5px
8px; font-size: 12px; background:
#ff4b5c;
}
li button:hover { background:
  #ff1e38;
}
```

SCRIPT.JS:

```
function add() { fetch("/products", {
method: "POST", headers: {"Content-
Type":"application/json"}, body:
JSON.stringify(getData())
  }).then(get);
}
function get() { fetch("/products")
  .then(res => res.json())
  .then(data => { let list =
    document.getElementById("list"); list.innerHTML
    = ""; data.forEach(p => { let li =
    document.createElement("li"); li.innerText =
    `${p.id} - ${p.name} - ₹${p.price}`;
    list.appendChild(li);
    });
```

```
   });
}
function getData() { return { id:
   document.getElementById("id").value, name:
   document.getElementById("name").value, price:
   document.getElementById("price").value
   };
}
```

SERVER.JS:

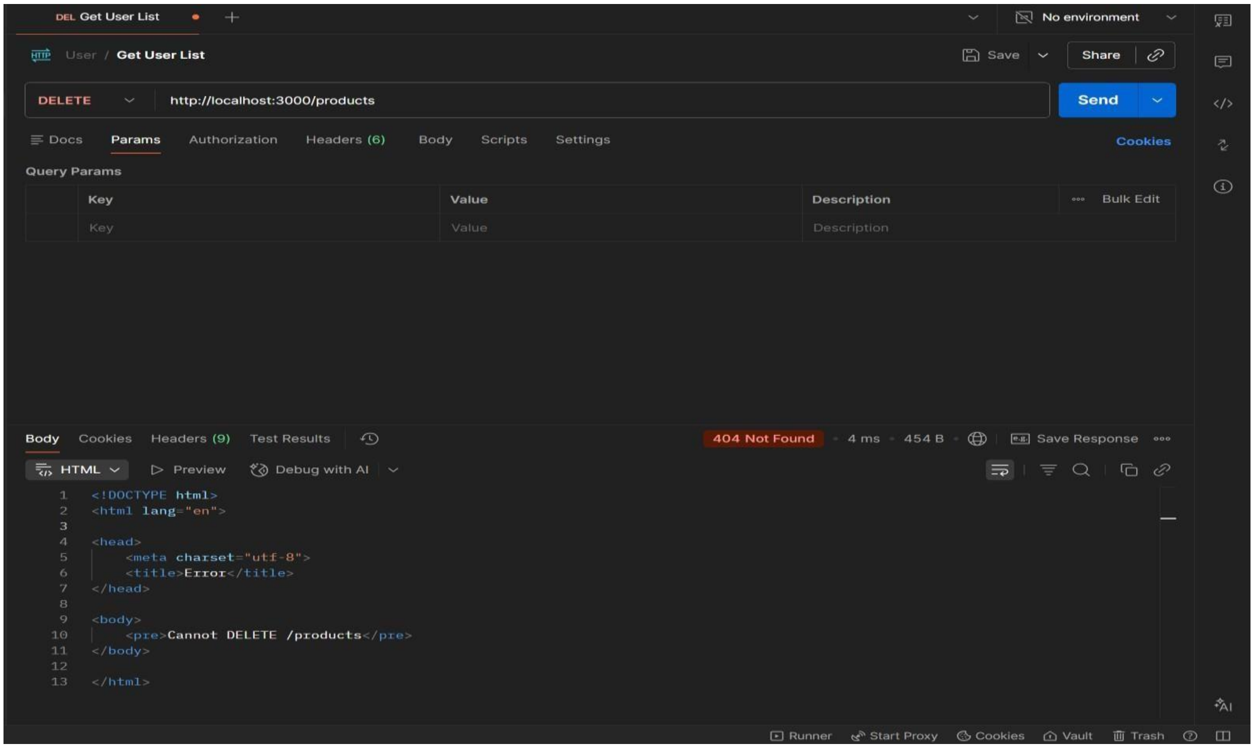
```
const express = require("express");
const cors = require("cors"); const
db = require("./db"); const app =
express(); app.use(cors());
app.use(express.json()); // GET all
products app.get("/products",
(req, res) => { res.json(db.getAll());
});
// POST add product
app.post("/products", (req, res) => {
db.add(req.body); res.json({ message:
"Product added" });
});
// PUT update product
app.put("/products/:id", (req, res) => {
```

```
db.update(req.params.id,            req.body);
res.json({ message: "Product updated" });
});
// DELETE product app.delete("/products/:id",
(req,   res)   =>   {   db.remove(req.params.id);
res.json({ message: "Product deleted" });
});
// Server start app.listen(3000, () => { console.log("    API running
at http://localhost:3000/products");
});
```

DB.JS:

```
let  products  =  [];
function  getAll()  {
return products;
}
function add(p) { products.push(p);
}
function update(id, newData) { products
   = products.map(p =>
      p.id == id ? { ...p, ...newData } : p
   );
}
function     remove(id)     {     products     =
   products.filter(p => p.id != id);
}
module.exports = { getAll, add, update, remove };
```

OUTPUT:



RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK14: MICROSERVICES APP

AIM: To design a simple microservices-based system by splitting a monolithic application into independent services with loose coupling and single responsibility.

ALGORITHM:

1. Start the program
2. Identify modules in monolithic system
3. Split system into independent services
4. Create separate folders for each service
5. Implement Service 1 (Product Service)
6. Implement Service 2 (Order Service)

7. Use separate databases (db.js) for each service
8. Assign different ports for each service
9. Enable communication via REST APIs
10. Keep services loosely coupled
11. Deploy and run services independently
12. Access services and verify functionality

PROGRAM:

INDEX.HTML:

```html
<!DOCTYPE html>

<html>

<head>

  <title>Microservices App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

  <h2>Microservices Demo</h2>

  <input id="pname" placeholder="Product Name">

  <button onclick="addProduct()">Add Product</button>

  <input id="order" placeholder="Order Item">

  <button onclick="addOrder()">Place Order</button>

</div>

<script src="script.js"></script>

</body>

</html>
```

STYLE.CSS:

```css
body { font-family: Arial;
```

```
background:    #1e1e2f;
color:                  white;
display:flex;        justify-
content:center;     align-
items:center;
height:100vh;
}
.container          {          text-
   align:center;
}
input  {  margin:
   5px; padding:
   8px;
}
button { padding:  8px;
   margin:              5px;
   background:
   #00c6ff;        border:
   none;
}
```

SCRIPT.JS:

```
function                        addProduct()                      {
   fetch("http://localhost:3001/products",             {
   method:   "POST",   headers:   {   "Content-Type":
   "application/json" }, body: JSON.stringify({ name:
   val("pname") })
```

```
  });
}
function addOrder() {
  fetch("http://localhost:3002/orders", { method:
  "POST", headers: { "Content-Type":
  "application/json" }, body: JSON.stringify({ item:
  val("order") })
  });
}
function val(id) { return
  document.getElementById(id).value;
}
```

SERVER.JS:

```
const express = require("express");
const cors = require("cors"); const
db = require("./db"); const app =
express(); app.use(cors());
app.use(express.json()); // GET
products app.get("/products",
(req, res) => { res.json(db.getAll());
});
// ADD product
app.post("/products", (req, res) => {
db.add(req.body); res.json({ msg:
"Product added" });
});
```

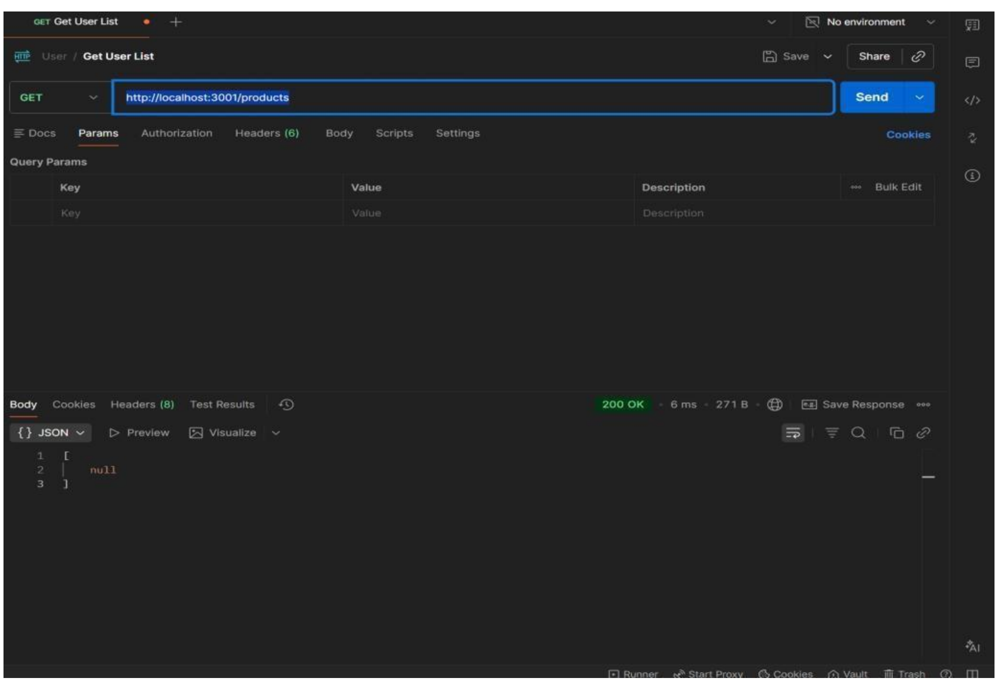
```
app.listen(3001, () => { console.log("Product Service running at
   http://localhost:3001");
});
```

DB.JS:

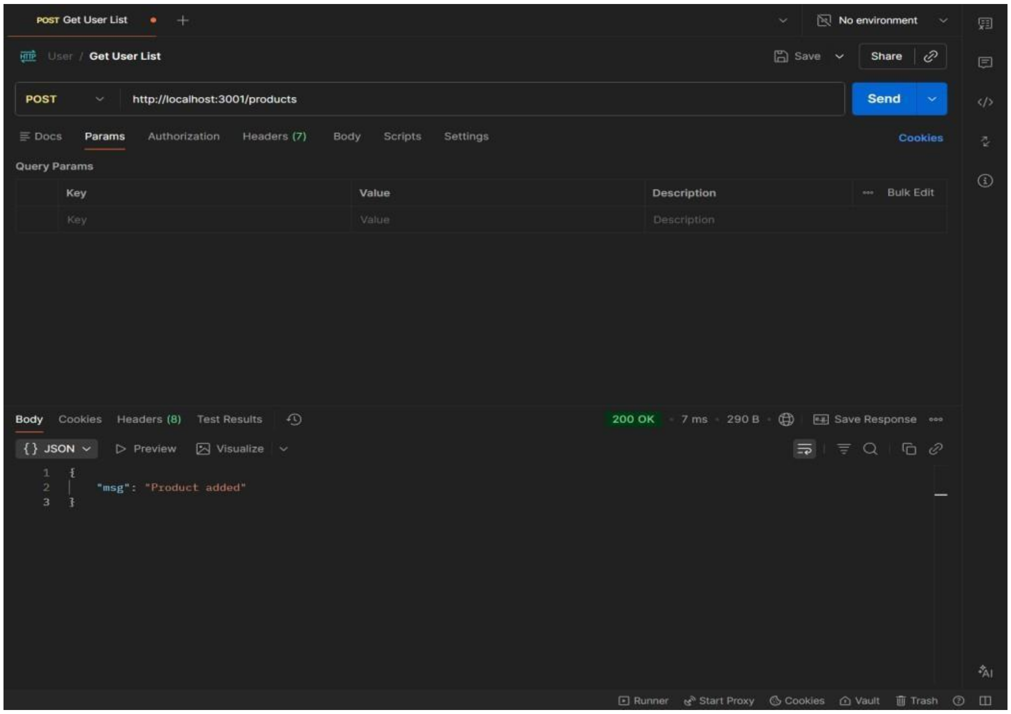
```
let products = [];
function add(p) {
products.push(p);
}
function getAll() { return
   products;
}
module.exports = { add, getAll };
```

OUTPUT:

1. **GET : http://localhost:3001/products**



2. POST: **http://localhost:3001/products**



RESULTS: The application was successfully divided into independent microservices with separate responsibilities and communication via REST APIs

TASK15: Service Registry App

Aim: To implement service registry and discovery using Node.js by dynamically registering and discovering microservices without hard-coded URLs.

ALGORITHM:

1. Start the program
2. Create a registry server
3. Store service name and URL in memory
4. Start Product Service
5. Register Product Service to registry
6. Start Order Service
7. Register Order Service to registry

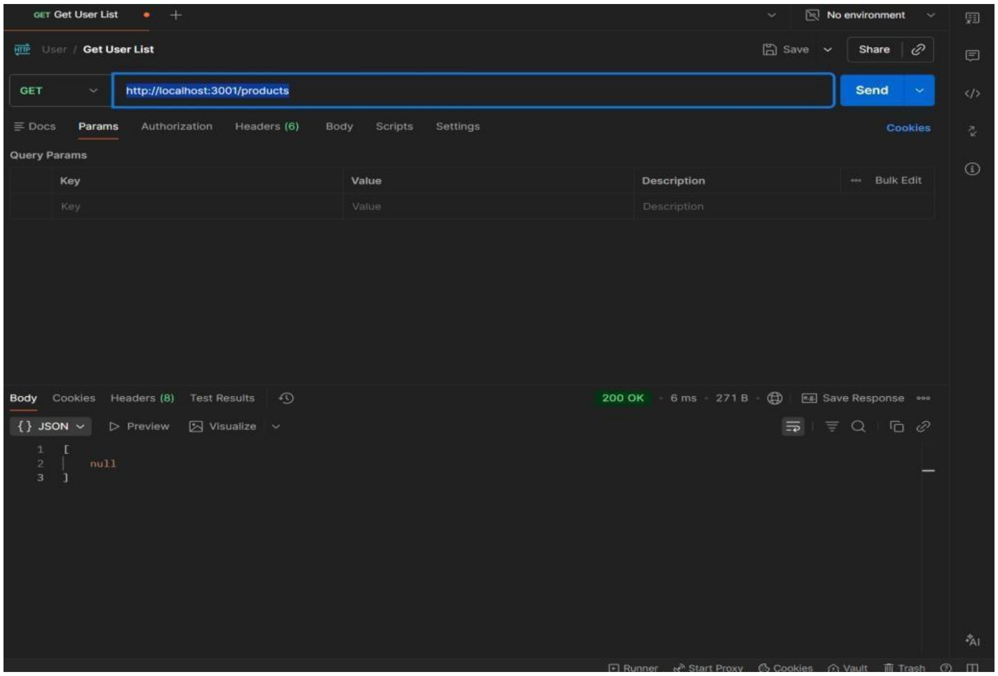
8. Maintain service list in registry
9. Client requests service from registry
10. Fetch service URL dynamically
11. Call the discovered service API
12. Stop the program

PROGRAM:

SERVER.JS:

```
const express =
require("express"); const app =
express(); app.use(express.json());
let services = {};
app.post("/register", (req, res) => {
const { name, url } = req.body;
services[name] = url;
console.log(`${name} registered at
${url}`); res.json({ msg:
"Registered" });
});
app.get("/service/:name", (req, res) => { const
  service = services[req.params.name];
  if (!service) { return res.status(404).json({ error: "Service
  not found" }) res.json({ url: service });
});
app.listen(4000, () => { console.log("Registry running at
  http://localhost:4000");
});
```

OUTPUT



RESULT: The microservices-based system was successfully implemented with service registry and discovery, enabling dynamic communication between independent services without hardcoded URLs.

TASK16: API GATEWAY API

AIM: To configure an API Gateway that routes client requests to microservices and performs basic load balancing across multiple service instances.

ALGORITHM:

1. Start the program
2. Create API Gateway server
3. Define routes for microservices
4. Create multiple instances of a service
5. Store service URLs in an array
6. Implement round-robin load balancing
7. Receive client request at gateway

8. Select service instance dynamically
9. Forward request to selected instance
10. Get response from service
11. Send response back to client
12. Stop the program

PROGRAM:

SERVER.JS:
```
const express =
require("express"); const axios =
require("axios"); const app =
express(); const services = [
  "http://localhost:3001",
  "http://localhost:3002"
];
let index = 0; app.get("/products", async
(req, res) => {
  try { const service = services[index]; index = (index + 1) %
    services.length; const response = await
    axios.get(service + "/products");
    res.json(response.data);

  } catch (err) { res.status(500).json({ error: "Service
    unavailable" });
  }
});
app.listen(5000, () => { console.log("API Gateway →
  http://localhost:5000/products");
```

});

OUTPUT:

```
{
  "instance": "Service 1",
  "data": ["Laptop", "Phone"]
}
```

```
{
  "instance": "Service 2",
  "data": ["Tablet", "Watch"]
}
```

RESULT: The API Gateway was successfully implemented to route requests and distribute load across multiple service instances using round-robin load balancing.

TASK17: Inter Service App

AIM: To enable inter-service communication where one microservice consumes another service's REST API and handles responses and failures gracefully.

ALGORITHM:

1. Start the program
2. Create Product Service (provider)
3. Create Order Service (consumer)
4. Implement REST API in Product Service
5. Store product data in memory
6. Implement Order Service API
7. Use axios/fetch to call Product Service
8. Receive response from Product Service

9. Process and store order data

10. Handle errors using try-catch

11. Send proper success/error response

12. Stop the program

FOLDER STRUCTURE:

```
inter-service-app/
|
├── product-service/
|  └── server.js
|
├── order-service/
|  └── server.js
```

PROGRAM:

PRODUCT-SERVICE:SERVER.JS:

```
const express = require("express");
const app = express(); let products
= [
  { id: 1, name: "Laptop" },
  { id: 2, name: "Phone" }
];
app.get("/products", (req, res) => { res.json(products);
});
app.listen(3001,   ()   =>   {   console.log("Product   Service   →
  http://localhost:3001/products");
```

```
});
```

ORDER-SERVICE:SERVER.JS:

```
const express = require("express");
const axios = require("axios"); const
app = express();
app.use(express.json()); let orders = [];
app.post("/orders", async (req, res) => {
try {
    const response = await axios.get("http://localhost:3001/products");
    const products = response.data; const order = { id: Date.now(),
      products: products
    };
    orders.push(order); res.json({ message:
    "Order created successfully", order:
    order
    });
  } catch (error) { res.status(500).json({ error: "Failed to
    fetch products from Product Service"
    });
  }
});
app.get("/orders", (req, res) => { res.json(orders);
});
app.listen(3002, () => { console.log("Order Service →
  http://localhost:3002/orders");
});
```

OUTPUT:

```
{
  "message": "Order created successfully",
  "order": {
    "id": 123456,
    "products": [
      { "id": 1, "name": "Laptop" },
      { "id": 2, "name": "Phone" }
    ]
  }
}
```

```
[
    {
        "id": 1,
        "name": "Laptop"
    },
    {
        "id": 2,
        "name": "Phone"
    }
]
```

RESULT: Inter-service communication was successfully implemented using REST APIs, with proper handling of responses and failures between microservices.

TASK18: Microservice Test

AIM: To write unit test cases for microservices to validate service logic, REST APIs, and data handling using a testing framework**.**

ALGORITHM:

1. Start the program
2. Install testing framework (Jest, Supertest)
3. Configure test script in package.json
4. Create test folder/files
5. Import service modules
6. Write test cases for service logic
7. Write test cases for REST APIs
8. Mock external dependencies if needed
9. Execute test cases using Jest
10. Verify expected outputs
11. Handle edge cases and errors
12. Stop the program

PROGRAM:

app.js:

```
const express = require("express");

const app = express();

app.use(express.json()); let

products = [];

// API app.get("/products", (req,

res) => { res.json(products);

});
```

```
app.post("/products", (req, res) => {
   products.push(req.body);
   res.json({ msg: "Product added" });
});
module.exports = app;
```

SERVER.JS:

```
const app = require("./app"); app.listen(3001,
() => {
   console.log("Product Service running");
});
```

Product.Test.js:

```
const request = require("supertest"); const app
= require("../product-service/app"); describe("Product API
Tests", () => { test("GET /products should return empty array",
async () => { const res = await request(app).get("/products");
expect(res.statusCode).toBe(200);
expect(res.body).toEqual([]);
   });
   test("POST /products should add product", async () => { const
      res = await request(app)
         .post("/products")
         .send({ name: "Laptop" });
      expect(res.statusCode).toBe(200);
      expect(res.body.msg).toBe("Product
      added");
   });
   test("GET /products should return data", async () => {
```

```
    const res = await request(app).get("/products");
    expect(res.body.length).toBeGreaterThan(0);
  });
});
```

OUTPUT:

```
PASS  tests/product.test.js
✓ GET /products should return empty array
✓ POST /products should add product
✓ GET /products should return data
```

RESULT: Unit test cases were successfully implemented to validate microservice logic, REST APIs, and data handling, ensuring application reliability.

TASK19: Mini Git System

AIM: To design a simple Git-like web application that tracks file changes, stores versions, and displays commit history using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for entering file content
4. Capture user input using JavaScript
5. Send data to server using fetch API
6. Store versions in database (array)
7. Assign version number to each commit
8. Save commit message and timestamp
9. Create API to fetch commit history
10. Display commit history in UI
11. Allow viewing previous versions
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

   <title>Mini Git System</title>

   <link rel="stylesheet" href="style.css">

</head>

<body>
```

```
<div class="container">
  <h2>Mini Git System</h2>
  <textarea id="content" placeholder="Write your code..."></textarea>
  <input id="message" placeholder="Commit message">
  <button onclick="commit()">Commit</button>
  <h3>Commit History</h3>
  <ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-family: Arial;
background:    #1e1e2f;
color:                  white;
display:flex;        justify-
content:center;     align-
items:center;
height:100vh;
}
.container  {  width:
  400px; text-align:
  center;
}
textarea { width:
  100%;  height:
```

```
  80px;  margin:
  5px;
}
input,  button  {
  margin:   5px;
  padding: 8px;
} li
{
  background: #333;
  margin:          5px;
  padding: 5px;
}
```

SCRIPT.JS:

```
function        commit()        {        const        content        =
  document.getElementById("content").value; const message =
  document.getElementById("message").value;
  fetch("/commit",  {  method:  "POST",  headers:  {"Content-
  Type":"application/json"},   body:   JSON.stringify({   content,
  message })
  }).then(loadHistory);
}
function             loadHistory()             {
  fetch("/history")
  .then(res => res.json())
  .then(data => {
```

```
        let list = document.getElementById("history");
        list.innerHTML = ""; data.forEach(c => { let li =
        document.createElement("li");  li.innerText  =
        `v${c.version}:                        ${c.message}`;
        list.appendChild(li);
        });
    });
}
loadHistory();
```

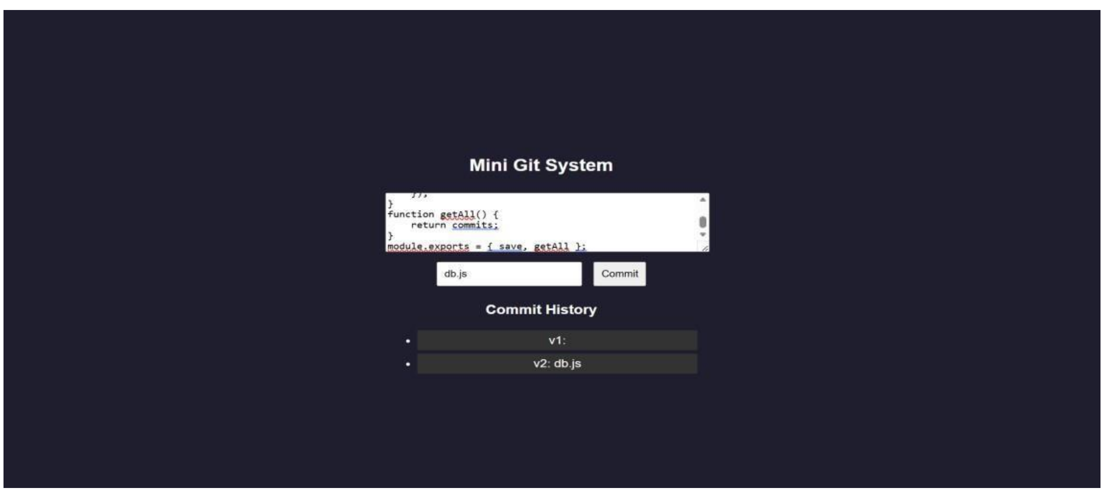
SERVER.JS:

```
const express = require("express");
const db = require("./db"); const
app = express();
app.use(express.json());
app.use(express.static(_dirname));
app.post("/commit", (req, res) => {
db.save(req.body); res.json({ msg:
"Committed" });
});
app.get("/history",       (req,       res)       =>       {
    res.json(db.getAll());
});
app.listen(3000,              ()              =>              {
    console.log("http://localhost:3000");
});
```

DB.JS:

```
let commits = []; let version
= 1; function save(data) {
commits.push({ version:
version++, message:
data.message, content:
data.content, time: new
Date()
  });
}
function getAll() { return
  commits;
}
module.exports = { save, getAll };
```

OUTPUT:



RESULT: A mini Git-like system was successfully developed to track versions, store commits, and display history through a web interface.

TASK20: Mini Git Branch App

Aim: To develop a Git-like application that supports branching, merging, rebasing, and conflict resolution using a web interface.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Store branches and commits in memory
4. Create default branch (main)
5. Add feature to create new branch
6. Commit changes to selected branch
7. Implement merge logic
8. Detect conflicts (same file modified)
9. Show conflict message
10. Resolve conflicts manually
11. Implement rebase logic
12. Display branch history and stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>Mini Git Branching</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
  <h2>Mini Git Branching</h2>
  <input id="branchName" placeholder="New Branch Name">
  <button onclick="createBranch()">Create Branch</button>
  <input id="message" placeholder="Commit Message">
  <button onclick="commit()">Commit</button>
  <button onclick="merge()">Merge</button>
  <button onclick="rebase()">Rebase</button>
  <h3>Branches</h3>
  <ul id="branches"></ul>
  <h3>History</h3>
  <ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-family: Arial;
background:    #1e1e2f;
color:                  white;
display:flex;        justify-
content:center;     align-
items:center;
height:100vh;
}
.container {
  text-align:center;
}
```

```
input,  button  {
   margin:   5px;
   padding: 8px;
} li
{
   background: #333;
   margin:          5px;
   padding: 5px;
}
```

SCRIPT.JS:

```
function        createBranch()        {        const        name        =
   document.getElementById("branchName").value;
   fetch("/branch",   {   method:   "POST",   headers:   {"Content-
   Type":"application/json"}, body: JSON.stringify({ name })
   }).then(load);
}
function        commit()        {        const        message        =
   document.getElementById("message").value;
   fetch("/commit",   {   method:   "POST",   headers:   {"Content-
   Type":"application/json"}, body: JSON.stringify({ message })
   }).then(load);
}
function  merge()  {  fetch("/merge",  {
   method: "POST" })
   .then(res => res.json())
   .then(alert)
```

```
   .then(load);
}
function rebase() { fetch("/rebase", {
   method: "POST" })
   .then(load);
}
function          load()          {
   fetch("/data")
   .then(res => res.json())
   .then(data       =>       {       let       b       =
      document.getElementById("branches"); let h =
      document.getElementById("history");
      b.innerHTML = "";
      h.innerHTML             =             "";
      data.branches.forEach(br => { let li =
      document.createElement("li");
      li.innerText = br;
         b.appendChild(li);
      });
data.history.forEach(c => {
         let li = document.createElement("li"); li.innerText
         = c;
         h.appendChild(li);
      });
   });
}
```

load();

SERVER.JS:

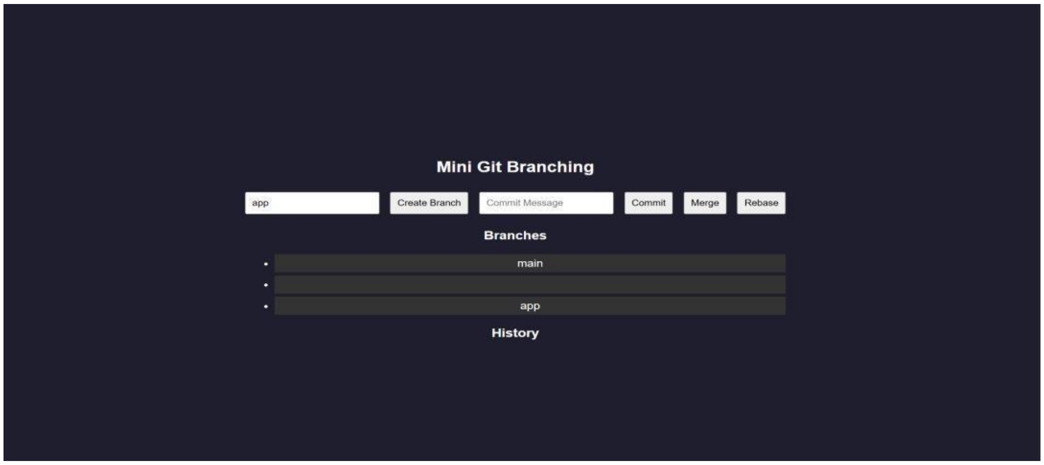
```
const express = require("express");
const db = require("./db"); const app
= express(); app.use(express.json());
app.use(express.static(_dirname));
app.post("/branch", (req, res) => {
db.createBranch(req.body.name);
res.json({ msg: "Branch created" });
});
app.post("/commit", (req, res) => {
  db.commit(req.body.message);
  res.json({ msg: "Committed" });
});
app.post("/merge", (req, res) => {
  const   result   =   db.merge();
  res.json({ msg: result });
});
app.post("/rebase", (req, res) => { db.rebase(); res.json({ msg: "Rebased" });
});
app.get("/data", (req, res) => { res.json(db.getData());
});
app.listen(3000, () => console.log("http://localhost:3000"));
```

DB.JS:

```
let branches = ["main"];
let current = "main"; let
commits = { main: []
};
function createBranch(name) {
  branches.push(name); commits[name]
  = [...commits[current]];
}
function commit(msg) {
  commits[current].push(msg);
}
function merge() { let mainCommits =
commits["main"]; let featureCommits =
commits[current]; if (mainCommits.length &&
featureCommits.length) { return "Merge Conflict
Detected!";
  }
commits["main"] = [...mainCommits, ...featureCommits]; return
  "Merged Successfully";
}
function rebase() { commits[current] = [...commits["main"],
  ...commits[current]];
}
function getData() { return {
  branches, history:
  commits[current]
```

```
  };
}
module.exports = { createBranch, commit, merge, rebase, getData };
```

OUTPUT:



RESULT: A Git-like branching application was successfully developed to simulate branching, merging, rebasing, and conflict resolution through a web interface.

TASK21: Mini Cicd App

AIM: To develop a CI/CD pipeline simulation application that automates build, test, and deployment steps through a web interface.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for pipeline trigger
4. Capture user action (Run Pipeline)
5. Send request to backend
6. Simulate code checkout step
7. Simulate build process

8. Simulate test execution
9. Simulate deployment step
10. Store pipeline logs
11. Display pipeline status in UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
   <title>Mini CI/CD Pipeline</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
   <h2>CI/CD Pipeline Simulator</h2>
   <button onclick="runPipeline()">Run Pipeline</button>
   <h3>Pipeline Logs</h3>
   <ul id="logs"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-family: Arial;
background:    #1e1e2f;
color:                   white;
```

```
display:flex;         justify-
content:center;     align-
items:center;
height:100vh;
}
.container   {   text-
    align:center;
    width: 400px;
}
button { padding: 10px;
    margin:             10px;
    background:
    #00c6ff;         border:
    none;   color:   white;
    cursor: pointer;
} li
{
    background: #333;
    margin:           5px;
    padding: 5px;
}
```

SCRIPT.JS:

```
function runPipeline() { fetch("/run",
    { method: "POST" })
    .then(() => loadLogs());
}
```

```
function loadLogs() {
  fetch("/logs")
  .then(res => res.json())
  .then(data => { let list =
    document.getElementById("logs");
    list.innerHTML = ""; data.forEach(log => {
    let li = document.createElement("li");
    li.innerText = log; list.appendChild(li);
    });
  });
}
loadLogs();
```

SERVER.JS:

```
const express = require("express");
const db = require("./db"); const app =
express(); app.use(express.json());
app.use(express.static(_dirname));
app.post("/run", async (req, res) => {
await db.runPipeline(); res.json({ msg:
"Pipeline executed" });
});
app.get("/logs", (req, res) => {
  res.json(db.getLogs());
});
app.listen(3000, () => {
  console.log("http://localhost:3000");
```

```
});
```

DB.JS: let logs = []; async function

```
runPipeline() { logs = []; logs.push("
Code Checkout..."); await delay();
logs.push("  Build Started..."); await
delay();
  logs.push("  Running Tests..."); await
  delay(); logs.push("  Tests Passed");
  await delay(); logs.push("•   Deploying
  Application..."); await delay();
  logs.push("   Deployment Successful!");
}
function getLogs() { return
  logs;
}
function delay() { return new Promise(resolve =>
  setTimeout(resolve, 1000));
}
module.exports = { runPipeline, getLogs };
```

OUTPUT:

RESULT: A CI/CD pipeline simulation application was successfully developed to automate and visualize build, test, and deployment processes.

TASK22: Cloud App

AIM: To develop a cloud comparison application that displays and compares services and pricing models of AWS, GCP, and Azure

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design user interface with dropdown for cloud providers
4. Initialize cloud service data (AWS, GCP, Azure) in database module
5. Capture selected provider using JavaScript
6. Send request to server to fetch service details
7. Display compute, storage, database, and networking services
8. Accept user input for compute hours and storage usage
9. Send input data to server for cost calculation
10. Calculate cost based on pay-as-you-use pricing model
11. Display total cost dynamically on UI

12. Stop the program

PROGRAM:

INDEX.HTML:

```html
<!DOCTYPE html>

<html>

<head>

  <title>Cloud Comparison</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

  <h2>Cloud Service Comparison</h2>

  <select id="provider" onchange="loadData()">

    <option value="aws">AWS</option>

    <option value="gcp">GCP</option>

    <option value="azure">Azure</option>

  </select>

  <h3>Services</h3>

  <ul id="services"></ul>

  <h3>Pricing Calculator</h3>

  <input id="hours" placeholder="Compute Hours">

  <input id="storage" placeholder="Storage (GB)">

  <button onclick="calculate()">Calculate Cost</button>

  <h3 id="result"></h3>

</div>

<script src="script.js"></script>
```

</body>

</html> **STYLE.CSS:**

```
body { font-family: Arial;
background: #1e1e2f;
color: white;
display:flex; justify-
content:center; align-
items:center;
height:100vh;

}
.container { text-
   align:center;
   width: 350px;
}
input, select, button {
   margin: 5px;
   padding: 8px;
} li
{
   background: #333;
   margin: 5px;
   padding: 5px;
}
```

SCRIPT.JS:

```
function loadData() { const provider =
  document.getElementById("provider").value;
  fetch(`/services/${provider}`)
  .then(res => res.json())
  .then(data => { let list =
    document.getElementById("services");
    list.innerHTML = "";
    Object.entries(data).forEach(([key, value]) => {
      let li = document.createElement("li");
      li.innerText = `${key}: ${value}`;
      list.appendChild(li);
    });
  });
}
function calculate() { const provider =
  document.getElementById("provider").value; const hours =
  document.getElementById("hours").value; const storage =
  document.getElementById("storage").value; fetch("/cost", {
  method: "POST", headers: {"Content-
  Type":"application/json"}, body: JSON.stringify({ provider,
  hours, storage })
  })
  .then(res => res.json())
  .then(data => { document.getElementById("result").innerText = "Cost:
    $" + data.cost;
  });
```

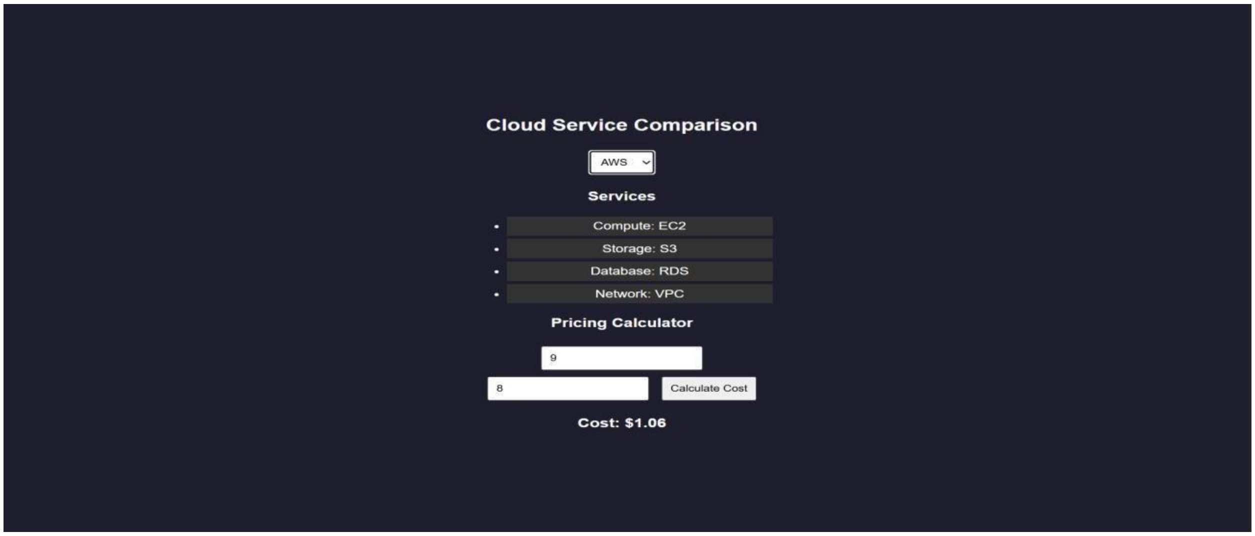
```
}
loadData(); SERVER.JS: const express =
require("express"); const db = require("./db");
const app = express(); app.use(express.json());
app.use(express.static(____________dirname));
app.get("/services/:provider", (req, res) => {
res.json(db.getServices(req.params.provider));

});
app.post("/cost", (req, res) => { const { provider,
  hours, storage } = req.body; const cost =
  db.calculate(provider, hours, storage); res.json({
  cost });
});
app.listen(3000, () => {
  console.log("http://localhost:3000");
});
```

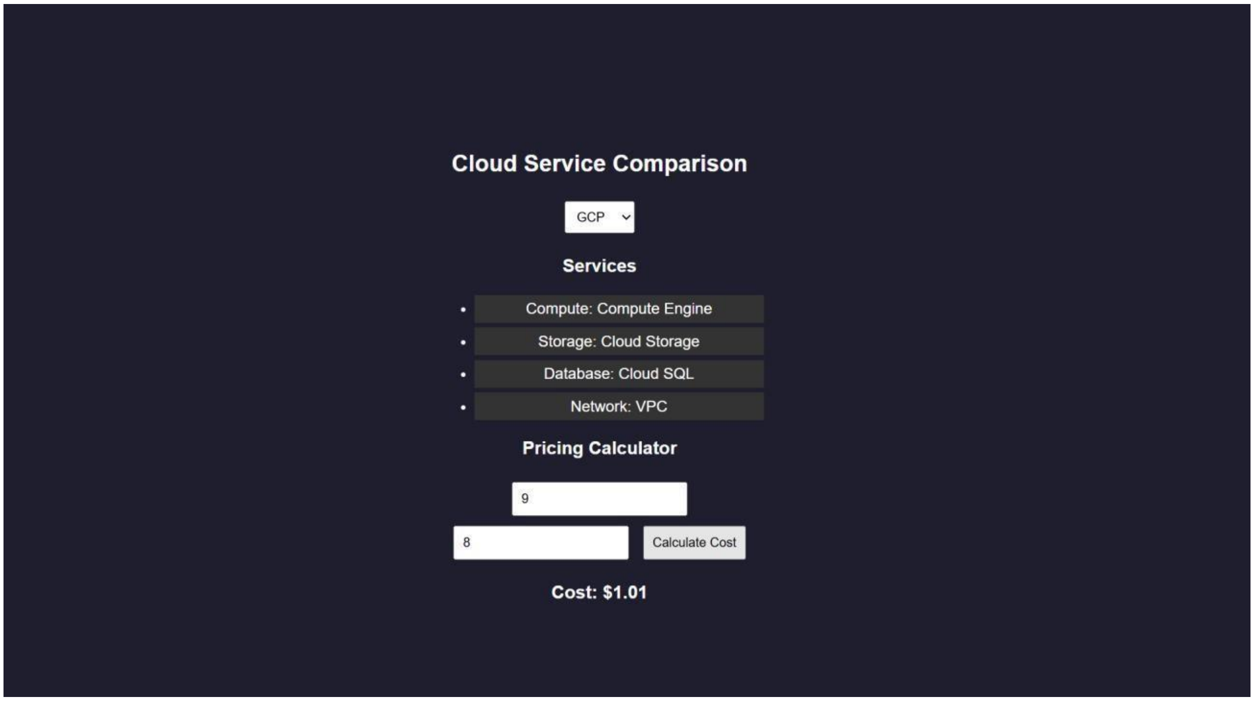
DB.JS:

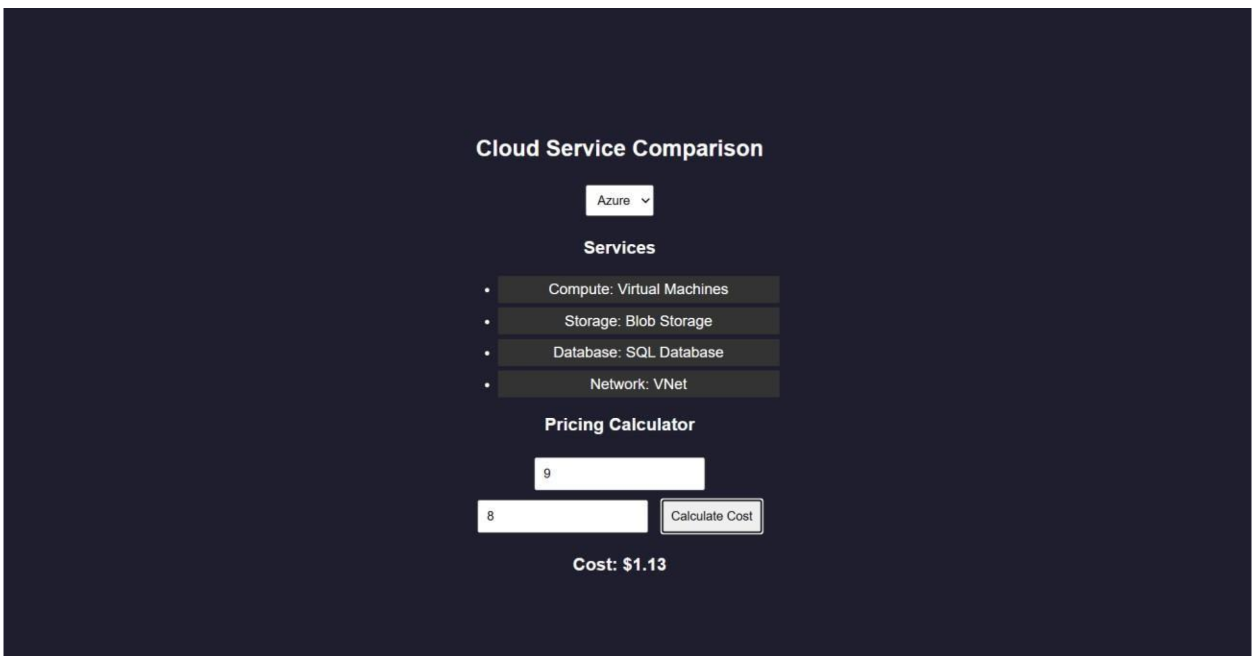
```
const services = { aws:
  {
    Compute: "EC2",
    Storage: "S3",
    Database: "RDS",
    Network: "VPC"
  },
  gcp: {
```

```
      Compute: "Compute Engine",
      Storage: "Cloud Storage",
      Database: "Cloud SQL",
      Network: "VPC"
  },
  azure: {
      Compute: "Virtual Machines",
Storage: "Blob Storage",
      Database: "SQL Database",
      Network: "VNet"
  }
};
const pricing = { aws: { compute: 0.1,
  storage: 0.02 }, gcp: { compute: 0.09,
  storage: 0.025 }, azure: { compute:
  0.11, storage: 0.018 }
};
function getServices(provider) { return
  services[provider];
}
function calculate(provider, hours, storage) { const p =
  pricing[provider]; return (hours * p.compute + storage *
  p.storage).toFixed(2);
}
module.exports = { getServices, calculate };
```

OUTPUT:


Cloud Service Comparison
AWS
Services
Compute: EC2
Storage: S3
Database: RDS
Network: VPC
Pricing Calculator
9
8
Calculate Cost
Cost: $1.06





RESULTS: A cloud comparison application was successfully developed to display services and simulate pay-as-you-use pricing across AWS, GCP, and Azure.

TASK23: AI PROMPT APP

AIM: To develop an application that demonstrates prompt engineering by generating and evaluating code or cloud configurations based on user input.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI with input field for user prompt
4. Capture user prompt using JavaScript
5. Send prompt to server using fetch API
6. Receive prompt data in backend (server.js)
7. Process prompt using predefined logic (db.js)
8. Generate output (code / cloud config / response)
9. Evaluate output quality and assign score
10. Send generated output and score to frontend
11. Display output and score on UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>AI Prompt App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
    <h2>Prompt Engineering Tool</h2>
    <textarea id="prompt" placeholder="Enter your prompt..."></textarea>
    <button onclick="generate()">Generate Output</button>
    <h3>Generated Output</h3>
    <pre id="output"></pre>
    <h3>Quality Score</h3>
    <p id="score"></p>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-family: Arial;
background:    #202038;
color:                 white;
display:flex;         justify-
content:center;     align-
items:center;
height:100vh;
}
.container  {  width:
   400px; text-align:
   center;
}
textarea { width:
   100%;  height:
```

```
100px;
margin: 5px;
}
button { padding: 10px;
margin:           10px;
background:
#00c6ff;       border:
none; color: white;
}
pre   {   background:
#333;      padding:
10px;
}
```

SCRIPT.JS:

```
function generate() { const prompt =
document.getElementById("prompt").value;
fetch("/generate", { method: "POST", headers: {"Content-
Type":"application/json"}, body: JSON.stringify({ prompt })
})
.then(res => res.json())
.then(data => {
document.getElem
entById("output").i
nnerText =
data.output;
document.getElem
```

entById("score").in

nerText =

data.score + "/10";

});

}

SERVER.JS: const express =

require("express"); const db =

require("./db");

const app = express();

// Middleware

app.use(express.json());

app.use(express.static(_dirname)); // serve HTML

// Home route (optional but useful)

app.get("/", (req, res) => { res.sendFile(

dirname + "/index.html");

});

// Generate output app.post("/generate", (req, res)

=> { const result =

db.processPrompt(req.body.prompt);

res.json(result);

});

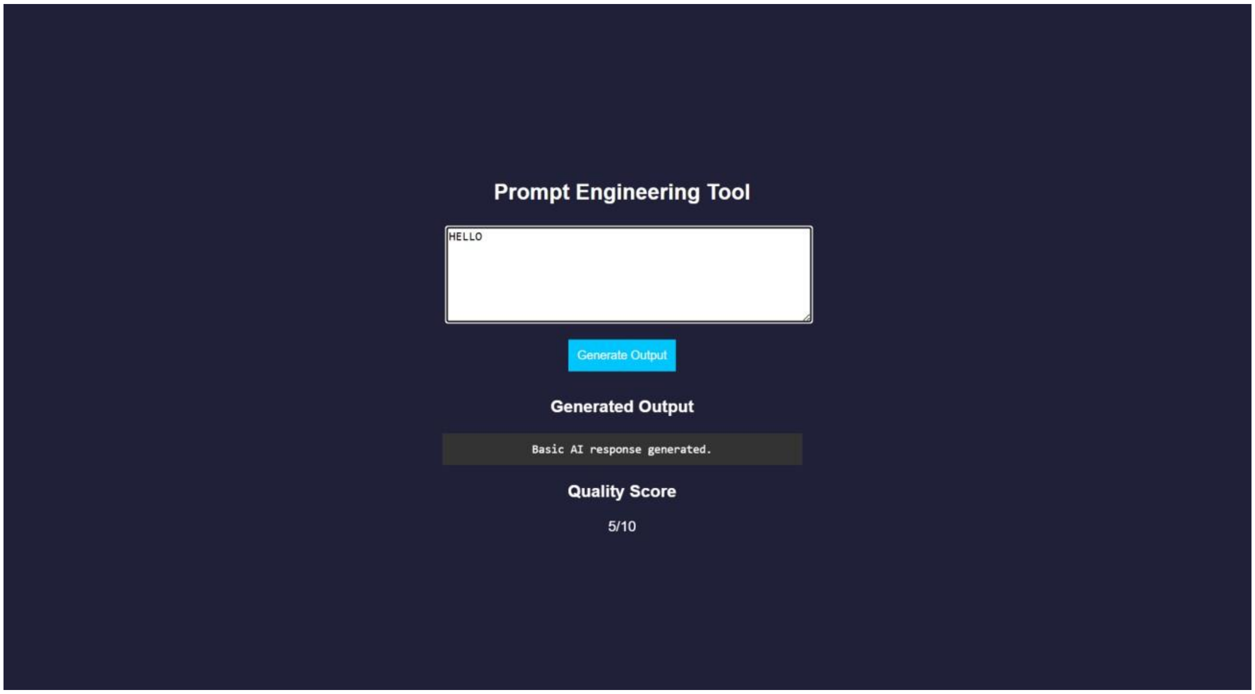
```
// START SERVER WITH LINK
app.listen(3000, () => { console.log("
Server running at:"); console.log("
http://localhost:3000");

});
```

DB.JS:

```
function processPrompt(prompt) { let output =
  ""; let score = 5; if
  (prompt.toLowerCase().includes("code")) {
  output = `function greet() {
  console.log("Hello from AI!");
}`;
    score = 9;
  }
  else if (prompt.toLowerCase().includes("cloud")) { output
    = `AWS EC2 Config:
- Instance: t2.micro
- Region: us-east-1`; score = 8;
  }
  else { output = "Basic AI response
    generated."; score = 5;
  }

  return { output, score };
}
```

module.exports = { processPrompt };

OUTPUT:



RESULT: A prompt engineering application was successfully developed to generate and evaluate AI-based outputs for code and cloud configurations.

TASK24: Vibe Cloud App

AIM: To develop an application that uses iterative prompt engineering to generate and evaluate REST APIs, cloud deployment steps, and security configurations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI to accept user prompts
4. Capture prompt input using JavaScript
5. Send prompt to backend using fetch API
6. Receive prompt in server and forward to processing module
7. Store prompt in history for version tracking
8. Analyze prompt type (REST API / Cloud / Security)
9. Generate corresponding output based on prompt
10. Evaluate output quality and assign score
11. Return output and score to frontend and display results
12. Stop the program

PROGRAM:
INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

   <title>Vibe Coding App</title>

   <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
   <h2>Vibe Coding Cloud Generator</h2>
   <textarea id="prompt" placeholder="Enter prompt..."></textarea>
   <button onclick="generate()">Generate</button>
   <h3>Output</h3>
   <pre id="output"></pre>
   <h3>Score</h3>
   <p id="score"></p>
   <h3>Prompt Versions</h3>
   <ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body { font-family: Arial;
background:    #b98bdb;
color:                   white;
display:flex;         justify-
content:center;      align-
items:center;
height:100vh;
}
.container  {  width:
   400px; text-align:
   center;
}
```

```
textarea { width:
   100%;  height:
   100px;
}
button { padding: 10px;
   margin:            10px;
   background:
   #00c6ff;        border:
   none;
}
pre    {    background:
   #333;       padding:
   10px;
} li
{
   background: #444;
   margin:           5px;
   padding: 5px;
}
```

SCRIPT.JS:

```
function        generate()        {        const        prompt        =
   document.getElementById("prompt").value;
   fetch("/generate", { method: "POST", headers: {"Content-
   Type":"application/json"},
```

```
    body: JSON.stringify({ prompt })
  })
  .then(res => res.json())
  .then(data => { document.getElementById("output").innerText =
    data.output; document.getElementById("score").innerText =
    data.score + "/10"; loadHistory();
  });
}
function loadHistory() {
  fetch("/history")
  .then(res => res.json())
  .then(data => { let list =
    document.getElementById("history");
    list.innerHTML = ""; data.forEach((p, i) => { let
    li = document.createElement("li"); li.innerText
    = `v${i+1}: ${p.prompt}`; list.appendChild(li);
    });
  });
}
loadHistory();
```

SERVER.JS:

```
const express =
require("express"); const db =
require("./db");

const app = express();
app.use(express.json());
```

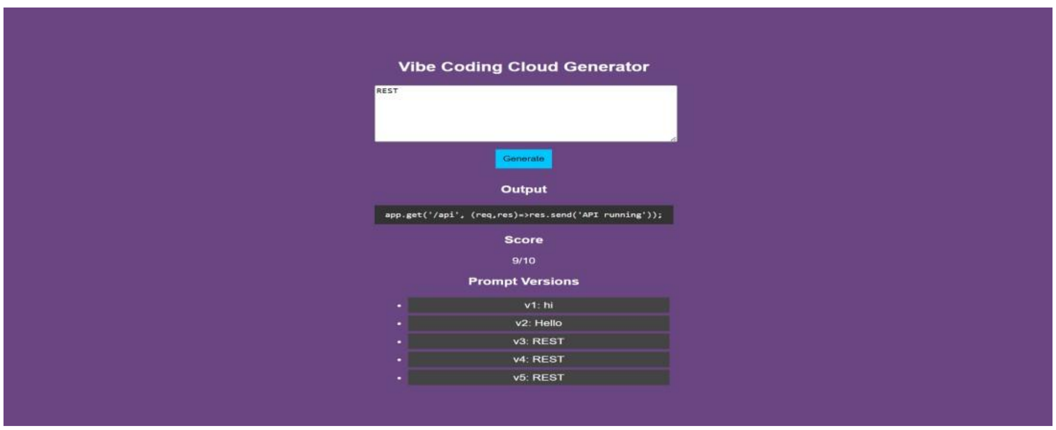
```
app.use(express.static(_dirname));

app.post("/generate", (req, res) => { const

result = db.process(req.body.prompt);

res.json(result);

});

app.get("/history",          (req,          res)          =>          {

   res.json(db.getHistory());

});

app.listen(3000,   ()   =>   {   console.log("•

   http://localhost:3000");

});
```

DB.JS:

```
let history = []; function process(prompt) { let output = ""; let

score = 5; history.push({ prompt }); if (prompt.includes("REST")) {

output  =  `app.get('/api',  (req,res)=>res.send('API  running'));`;

score = 7;

   }

   else if (prompt.includes("cloud")) { output

      = `Deploy using AWS EC2:

1. Launch instance

2. Install Node.js

3. Run app`; score = 8;

   }

   else if (prompt.includes("security")) { output = `Use HTTPS, JWT

      authentication, and firewall rules`; score = 9;

   }
```

```
   score += Math.min(history.length, 2); return
   { output, score };
}
function getHistory() { return
   history;
}
module.exports = { process, getHistory };
```

OUTPUT:



RESULT: A Vibe Coding application was successfully developed to generate and refine cloudbased features using iterative prompt engineering.